# DIY Neuroscience Kit Basic

Starter kit to explore brain, heart, muscle and eye signals

## Overview

This kit is perfect for students, researchers and hobbyists alike who want to start exploring neuroscience. Whether it's studying brain waves, monitoring heart rhythms, analyzing muscle movements, or tracking eye movements, this DIY kit provides an accessible and educational platform for understanding the complexities of human physiology and developing practical applications in the fields of human-computer interaction, and beyond.

## Kit Contents

- 2 Channel Brain BioAmp Band
- Muscle BioAmp Band
- Heart BioAmp Band
- BioAmp EXG Pill
- BioAmp V3 Cable (100 cm)
- Jumper Cables (x2)
- Arduino UNO EK R4 Minima with Type-C Cable
- NuPrep Gel (9g)
- Electrode Gel Pouch (100 ml)
- Alcohol Swabs (20 pcs)
- 12 Pcs Gel Electrodes
- Baby Gel Electrodes (100 pcs)

# Contents

**1 DIY Neuroscience Kit Basic**
1.1 Overview . . . 1
1.2 Kit Contents . . . 1
1.3 Software requirements . . . 4
1.4 Using the kit . . . 4
1.4.1 Step 1 (optional): Configure for EMG/ECG . . . 4
1.4.2 Step 2: Connect Arduino UNO EK R4 Minima . . . 5
1.4.3 Step 3: Connecting electrode cable . . . 6
1.4.4 Step 4: Skin Preparation . . . 6
1.4.5 Step 5: Electrode placement . . . 6
1.4.6 Step 6: Uploading the code . . . 7
1.4.7 Step 7: Setting up Chords Web . . . 7
1.4.8 Step 8: Setting up Chords Python . . . 7
1.5 Some Project Ideas . . . 8

**2 Using Gel Electrodes**
2.1 Overview . . . 9
2.2 Key components in a gel electrode . . . 9
2.3 Types of gel electrodes . . . 9
2.3.1 1. Classification on basis of water-content . . . 9
2.3.2 2. Classification on the basis of connectivity . . . 11
2.4 Using the electrodes . . . 13
2.4.1 1. Skin Preparation . . . 13
2.4.2 2. Connecting the cable . . . 13
2.4.3 3. Electrodes placement . . . 13
2.5 Removing the electrodes . . . 16

**3 Chords-Web**
3.1 Overview . . . 17
3.2 Quick Start . . . 17
3.3 Requirements . . . 18
3.3.1 Browser Compatibility . . . 18
3.3.2 Software Requirements . . . 19
3.3.3 Hardware Requirements . . . 19
3.3.4 Supported Boards . . . 19
3.4 Setting up the Hardware . . . 20
3.5 Uploading the Firmware . . . 20
3.5.1 Option 1: Update firmware from Chords-Web (Recommended) . . . 20
3.5.2 Option 2: Upload firmware with the Arduino IDE . . . 20
3.5.3 Option 3: NPG Lite and ESP32 boards . . . 21
3.6 Connecting to Chords-Web . . . 21
3.6.1 Connecting over Serial (USB) . . . 22
3.6.2 Connecting over Bluetooth (NPG Lite) . . . 22
3.7 Common Controls . . . 23
3.7.1 App Switcher . . . 23
3.7.2 Play/Pause and Data Windows . . . 24
3.7.3 Settings (Zoom, Time Base and Channels) . . . 24
3.7.4 Filters . . . 25
3.7.5 Recording the Data . . . 25
3.7.6 Disconnecting . . . 26
3.7.7 Other Options to Explore . . . 26
3.8 Applications . . . 26
3.8.1 Chords Visualizer . . . 26
3.8.2 FFT Visualizer . . . 29
3.8.3 Rep-Forge . . . 31

3.8.4 ECG Visualizer . . . 32
3.8.5 Serial Wizard Plotter & Monitor . . . 33
3.9 Data Format and Data Quality . . . 34
3.9.1 Data Format . . . 34
3.9.2 Checking for Dropped Samples . . . 35
3.10 Troubleshooting . . . 35
3.11 Technologies Used . . . 36

**4 Chords-Python**
4.1 Overview . . . 37
4.2 Features . . . 37
4.3 Software Requirements . . . 37
4.4 Hardware Requirements . . . 37
4.5 Setting up the hardware . . . 38
4.6 Uploading the code . . . 38
4.7 Opening Chords-Python . . . 38
4.7.1 A. Using the chordspy Package . . . 38
4.7.2 B. Running Scripts Manually (Alternative) . . . 40
4.8 Connection . . . 40
4.8.1 Wi-Fi . . . 41
4.8.2 Bluetooth . . . 41
4.8.3 Serial (USB) . . . 41
4.9 CSV Logging . . . 41
4.10 Applications . . . 42
4.10.1 1. *ECG with Heart Rate* . . . 42
4.10.2 2. *EMG with Envelope* . . . 43
4.10.3 3. *EOG with Blinks* . . . 46
4.10.4 Features . . . 47
4.10.5 4. *EEG with FFT* . . . 48
4.10.6 5. *EEG Tug of War Game* . . . 50
4.10.7 6. *EEG Beetle Game* . . . 51
4.10.8 7. *GUI* . . . 55
4.10.9 8. *EOG Keystroke Emulator* . . . 55
4.10.10 9. *CSV Plotter* . . . 58
4.10.11 10. *EOG Morse Decoder* . . . 59
4.10.12 Features . . . 61
4.11 Create Custom application . . . 64

# 1 DIY Neuroscience Kit Basic

## 1.3 Software requirements

To use your DIY Neuroscience Kit Basic, you will need the softwares mentioned below. Instructions on how to use them are provided later in the guide.

- Arduino IDE (Download this to upload Chords arduino firmware to your development board)
- Chords Web (Use this open-source web application to visualize your biopotential signals)
- Visual Studio Code (or any other Code editor of your choice)
- Python (To run Chords-Python script)
- Chords Python (Use this open-source python script designed to record and visualize biopotential signals)

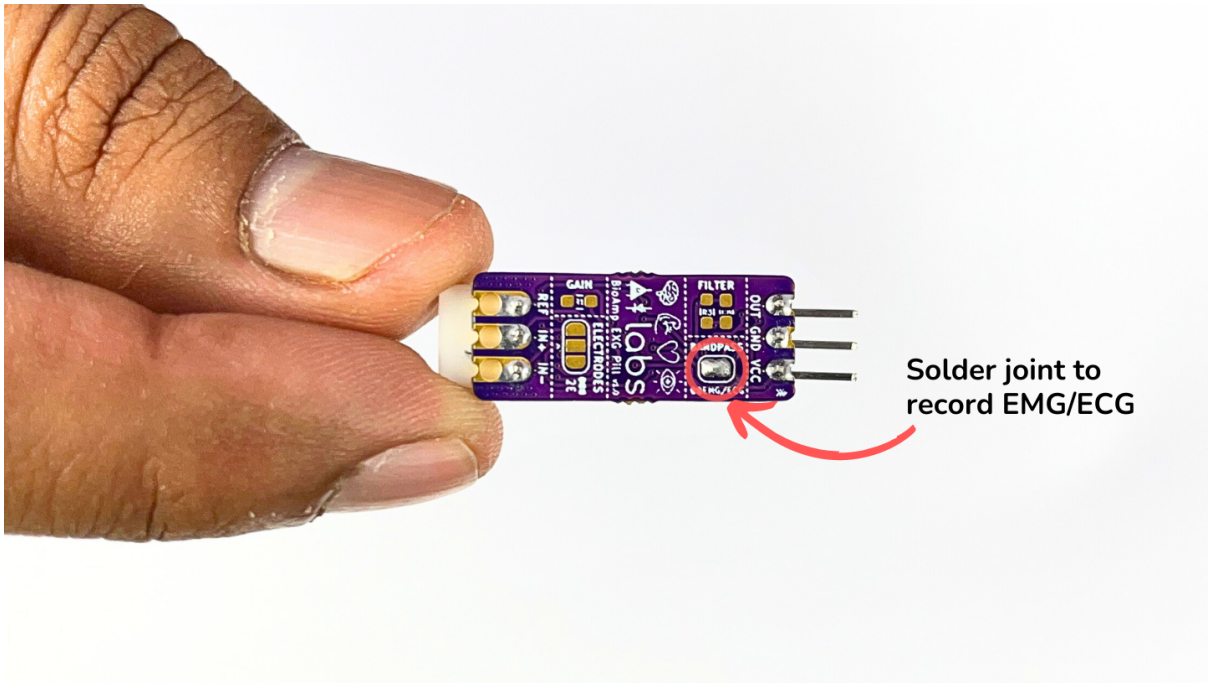
**Note**

1. The Chords Arduino firmware is identical for both Chords Web and Chords Python, so you only need to upload the code once, and you're all set.
2. You can choose either Chords Web or Chords Python to record and visualize your biopotential signals based on your needs. If you're curious, you can try both one at a time.

## 1.4 Using the kit

This kit is made in a way so that even beginners can use it and get started with recording biopotential signals from their body to explore the field of neuroscience by making HCI/BCI projects.

### 1.4.1 Step 1 (optional): Configure for EMG/ECG

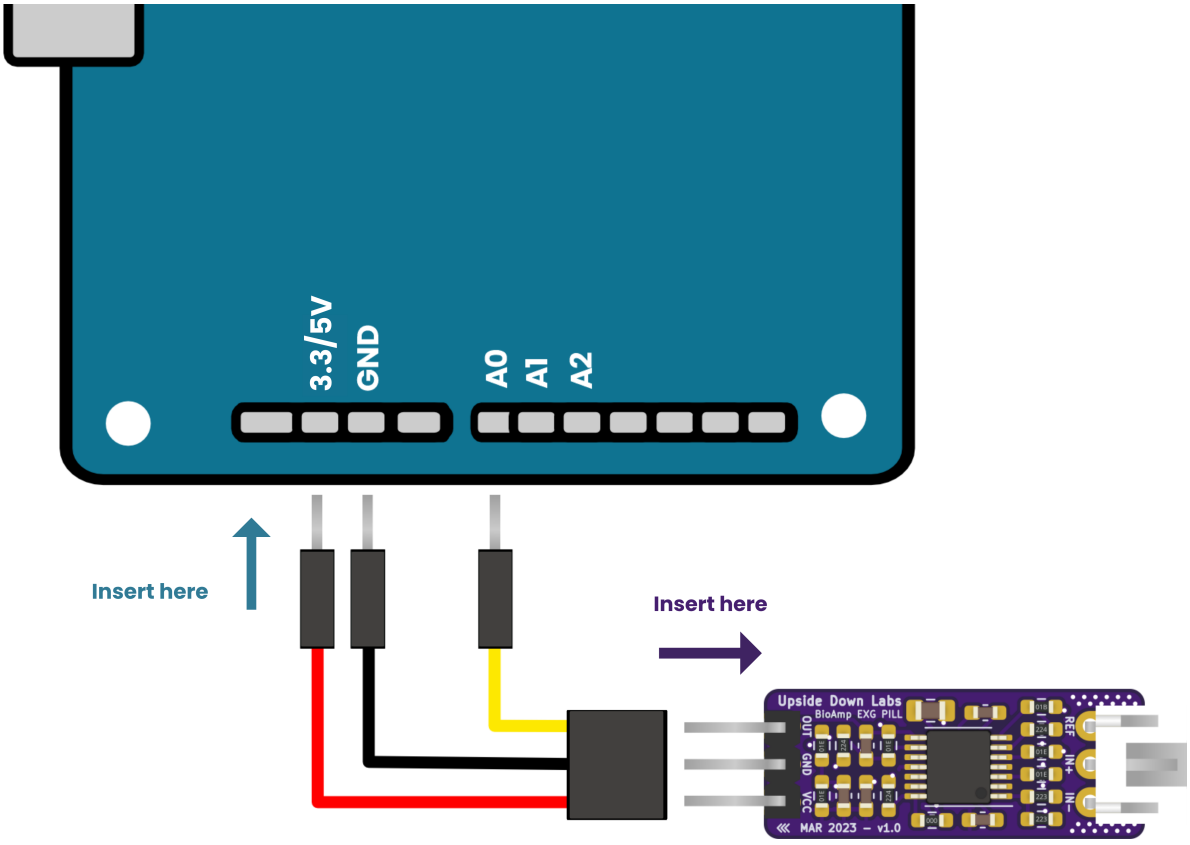


BioAmp EXG Pill is by default configured for recording EEG or EOG, so if you are recording any of the two signals you can skip this step. But if you want to record good quality ECG or EMG, then it is recommended to configure it by making a solder joint as shown in the image above.

**Note**

Even without making the solder joint the BioAmp EXG Pill is capable of recording ECG or EMG as well but the signals would be more accurate if you configure it.

### 1.4.2 Step 2: Connect Arduino UNO EK R4 Minima

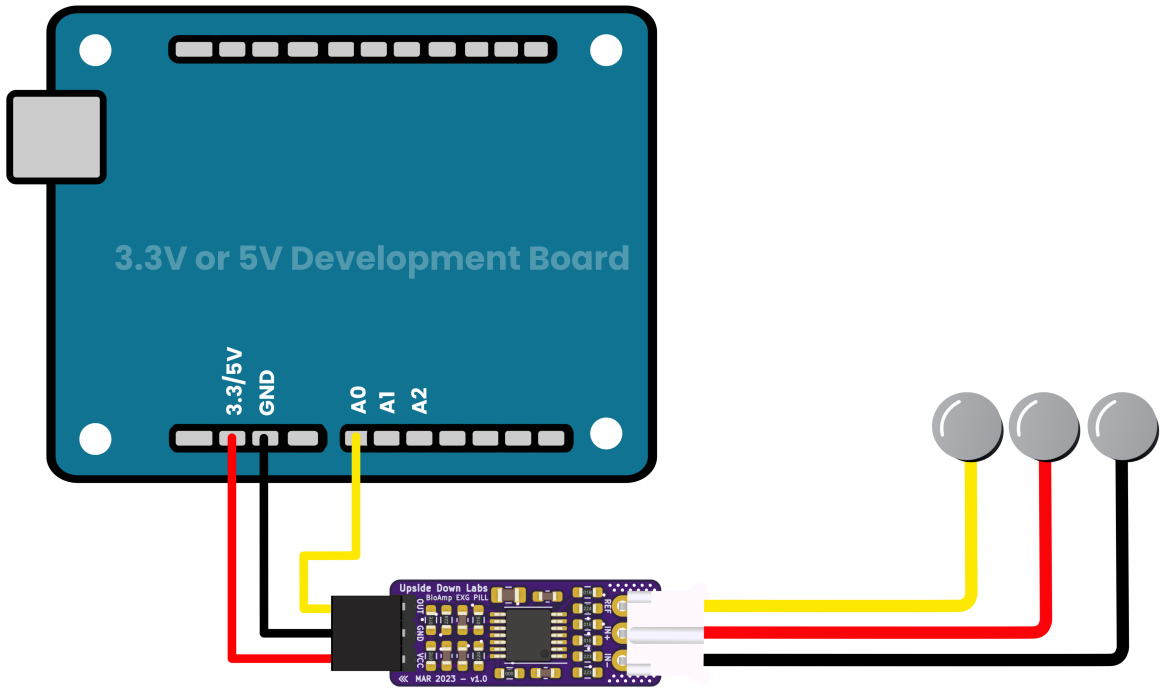



Connect `VCC` to `5V`, `GND` to `GND`, and `OUT` to `Analog pin A0` of your Arduino UNO EK R4 Minima via jumper cables provided by us. If you are connecting OUT to any other analog pin, then you will have to change the INPUT PIN in the Arduino sketch accordingly.

**Warning**

Take precautions while connecting to power, if power pins (GND & VCC) are to be swapped, your BioAmp EXG Pill will be fried and it'll become unusable (DIE).

### 1.4.3 Step 3: Connecting electrode cable




Connect the BioAmp cable to BioAmp EXG Pill by inserting the cable end in the JST PH connector as shown above.

### 1.4.4 Step 4: Skin Preparation

Apply Nuprep Skin Preparation Gel on the skin surface where electrodes would be placed to remove dead skin cells and clean the skin from dirt. After rubbing the skin surface thoroughly, clean it with an alcohol wipe or a wet wipe.

For more information, please check out detailed step by step Skin Preparation Guide.

### 1.4.5 Step 5: Electrode placement

We have 2 options to measure the signals, either using the gel electrodes or using dry electrode based BioAmp Bands. You can try both of them one by one.

1. *Using Gel electrodes guide*
2. Assembling & using BioAmp Bands guide

Once you have made the connections, return here to proceed to the next steps.

### 1.4.6 Step 6: Uploading the code

1. Connect the Arduino UNO EK R4 Minima to your laptop using the Type-C cable. Go to the Chords Arduino Firmware GitHub repository, open the `UNO-R4` folder, and copy the Arduino sketch into the Arduino IDE that you downloaded earlier.

   Link for the Arduino sketch: Chords Arduino Firmware for Arduino UNO R4

2. In Arduino IDE, open `Tools > Board > Boards Manager`. Search for `Arduino UNO R4 Boards` and install it.
3. Go to `Tools > Board > Arduino UNO R4 Boards` and select `Arduino UNO R4 Minima`.
4. Go to `Tools > Port` and select the COM port on which your Arduino UNO EK R4 Minima is connected. To find the correct COM port, disconnect the board and reopen the menu. The entry that disappears is the correct port. Now click the upload button.

**Warning**

Make sure your laptop is not connected to a charger and sit 5m away from any AC appliances for best signal acquisition.

### 1.4.7 Step 7: Setting up Chords Web

1. Visit chords.upsidedownlabs.tech.
2. Click on "Visualize now" button.
3. **At the bottom, you can see buttons to access various applications:**
   a. *Chords Visualizer*: Use this application for real-time data visualization, recording and data management, filter options, and multi-channel support.
   b. *FFT Visualizer*: Use this app to visualize filtered EEG signals in real-time, FFT graph, EEG frequency bands, and a beta candle to determine your focus.
   c. *Serial Wizard*: This interface provides real-time serial data visualization using serial plotter and monitor, optimised data rendering, baud rate selection and options to toggle between different modes.
4. Click on any of the button according to your requirement, select the COM port and click OK. You will be able to visualize your signals on the screen.

### 1.4.8 Step 8: Setting up Chords Python

Since you have uploaded the firmware already to your Arduino UNO EK R4 Minima, use our Python script and follow the steps given in the *Chords-Python documentation* for LSL streaming, CSV data logging, verbose output with detailed statistics and error reporting. Not only that, you get a complete web interface to access various applications (like ECG with heart rate, EMG with envelope, GUI of channels, CSV plotter, etc.) that you can use to further analyse your signals and create HCI/BCI projects.

## 1.5 Some Project Ideas

You can find step-by-step tutorials for various HCI/BCI projects on our Instructables.

Here are some project ideas that you can try making at your home. Click on the links below to get the step by step guides to build the projects.

1. Controlling video game using EEG signals
2. Recording EEG from visual cortex
3. Record publication-grade ECG signals
4. Measuring heart rate
5. Detecting heart beats
6. Creating a drowsiness detector
7. Detecting eye blinks
8. Detecting up and down movement of eyes
9. Recording publication-grade EMG signals

These are some of the project ideas but the possibilities are endless. So create your own Human Computer Interface (HCI) and Brain Computer Interface (BCI) projects and share them with us at contact@upsidedownlabs.tech

# 2 Using Gel Electrodes

## 2.1 Overview

Gel electrodes are type of electrodes that are applied on the skin surface to transmit the electrical signals present at the skin surface to a recording or monitoring device. These electrodes use a conductive hydrogel to improve the interface between the Ag-AgCl electrode and the skin, enhancing signal quality by reducing impedance and improving electrical contact. These are often used to record biopotential signals like Electrocardiography (ECG), Electroencephalography (EEG), Electromyography (EMG), and Electrooculography (EOG).

## 2.2 Key components in a gel electrode

`Electrode material`: The electrode itself is typically made of a conductive material such as Ag-AgCl, which is commonly used due to its stability and biocompatibility.

`Hydrogel`: A hydrogel is applied to the electrode or embedded within it. This gel helps maintain a consistent electrical connection with the skin by reducing the impedance at the interface.

`Backing`: Gel electrodes have various types of backing like foam, cloth, and cleartape.

`Adhesive`: Gel electrodes has a sticky glue like interface called adhesive that allows it to firmly attach to your skin, it can be of different types like hydrogel (where hydrogel acts both as a conductive gel and adhesive), standard, and aggressive.

## 2.3 Types of gel electrodes

### 2.3.1 1. Classification on basis of water-content

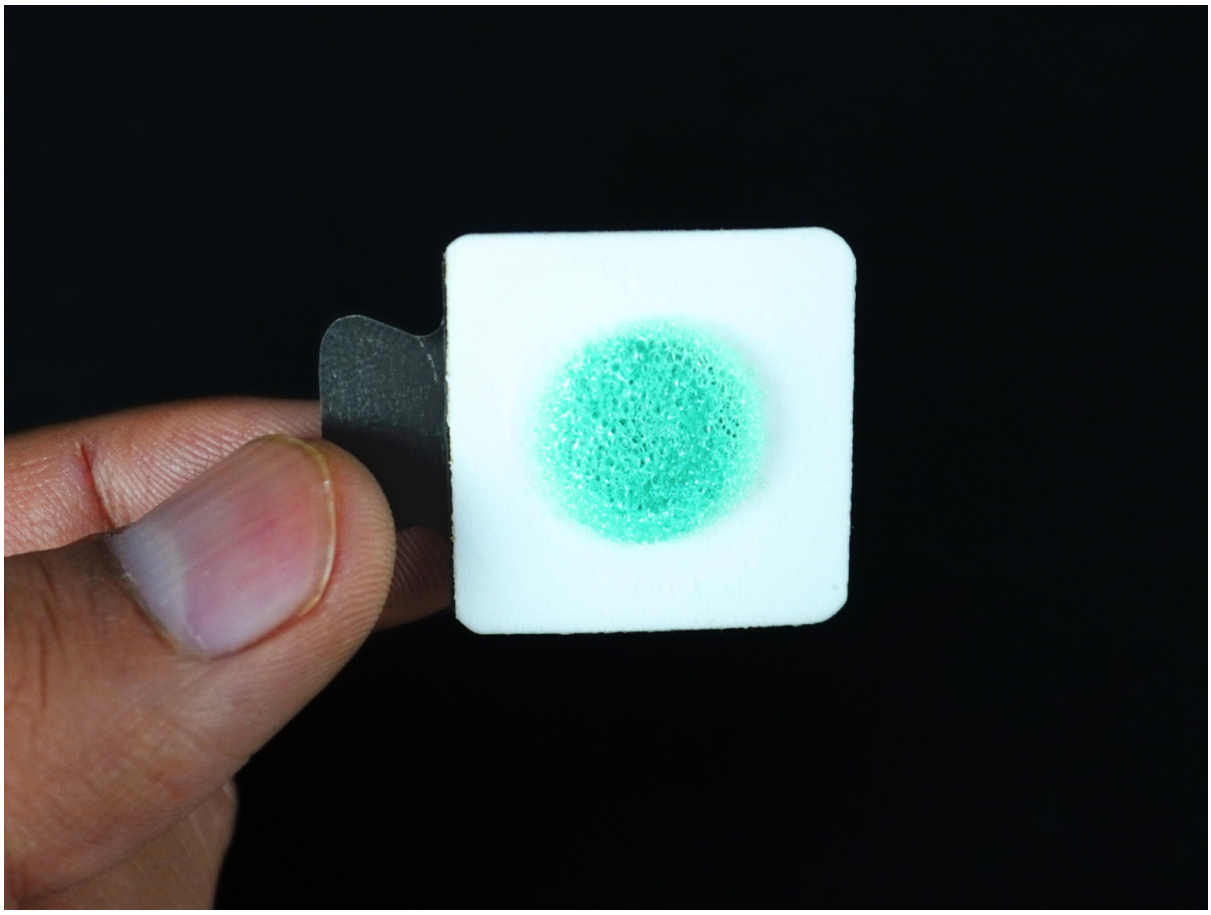
Below are the gel electrodes categorised on the basis of the water content present in the hydrogel:

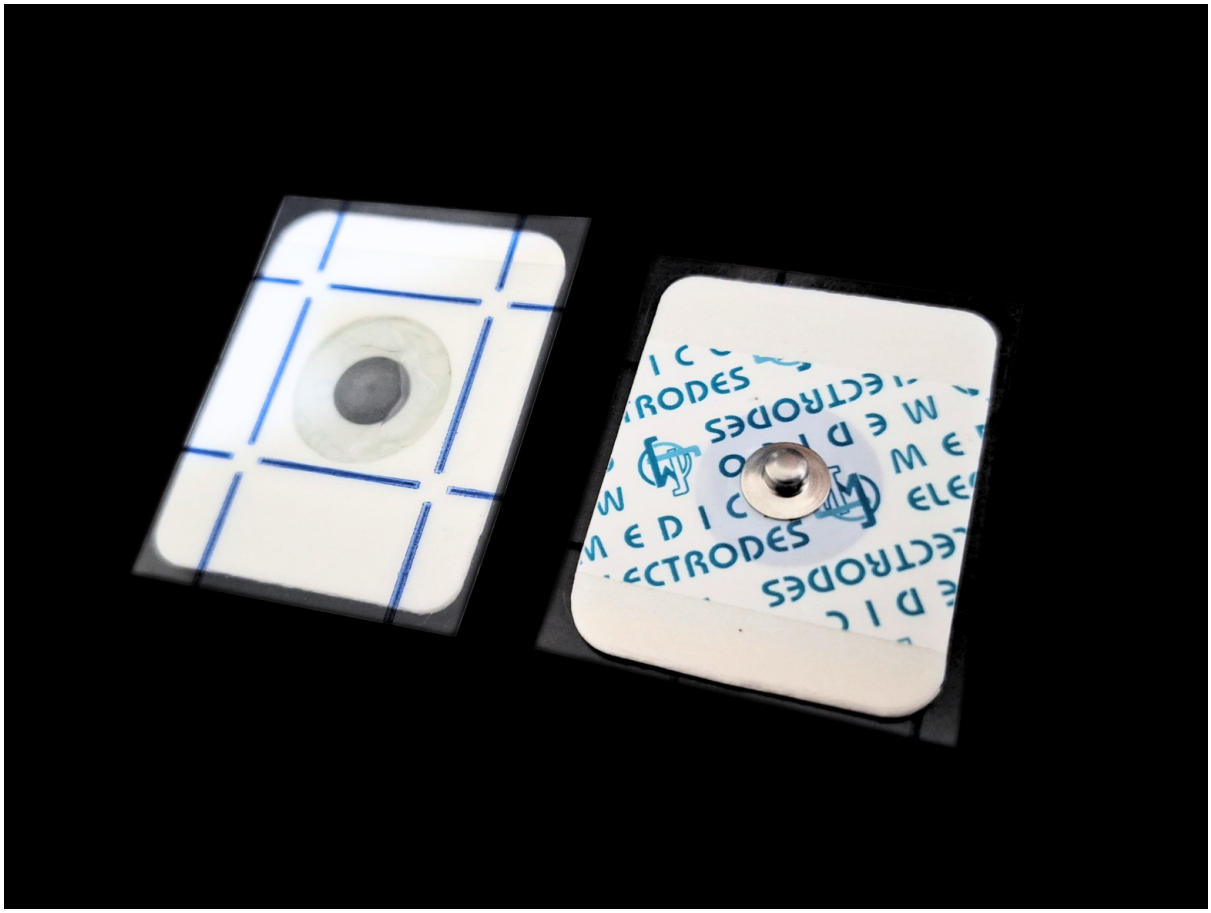
#### a. Liquid gel electrodes

These electrodes have hydrogel with higher water content which provide excellent conductivity but can dry out over time, reducing their effectiveness. They are preferred to be used if you have a dry skin and/or you want to record the biopotential data for a short time duration.

#### b. Solid gel electrodes

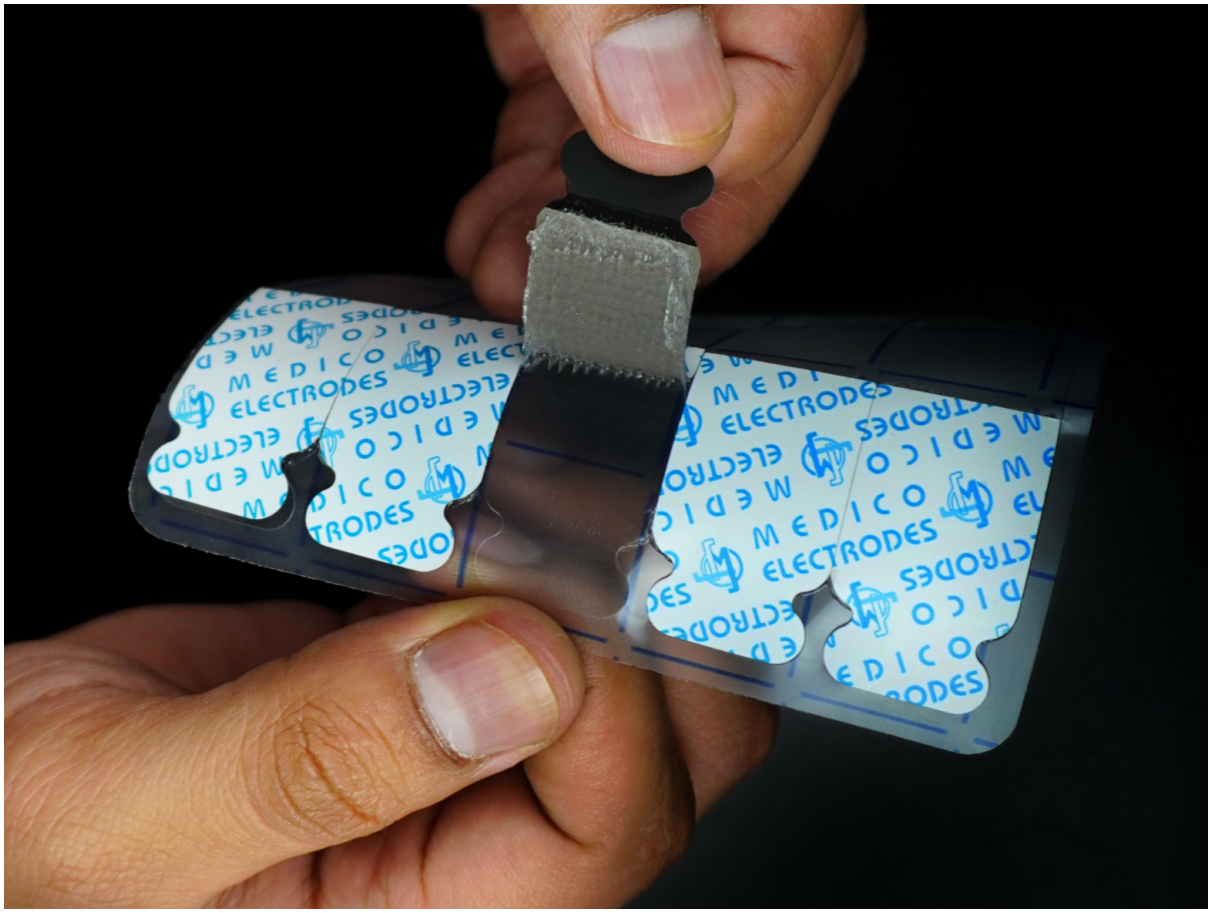
These electrodes contain hydrogel that is more solid or jelly-like. They are less prone to drying out as compared to liquid gel electrodes and have a strong adhesion.





#### c. Full-surface hydrogel electrodes

This is a type of solid gel electodes where hydrogel has minimal water content, covers the entire electode surface and acts as an adhesive. These electrodes are more comfortable for long-term monitoring. They offer good adhesion, conductivity and are less likely to cause skin irritation.



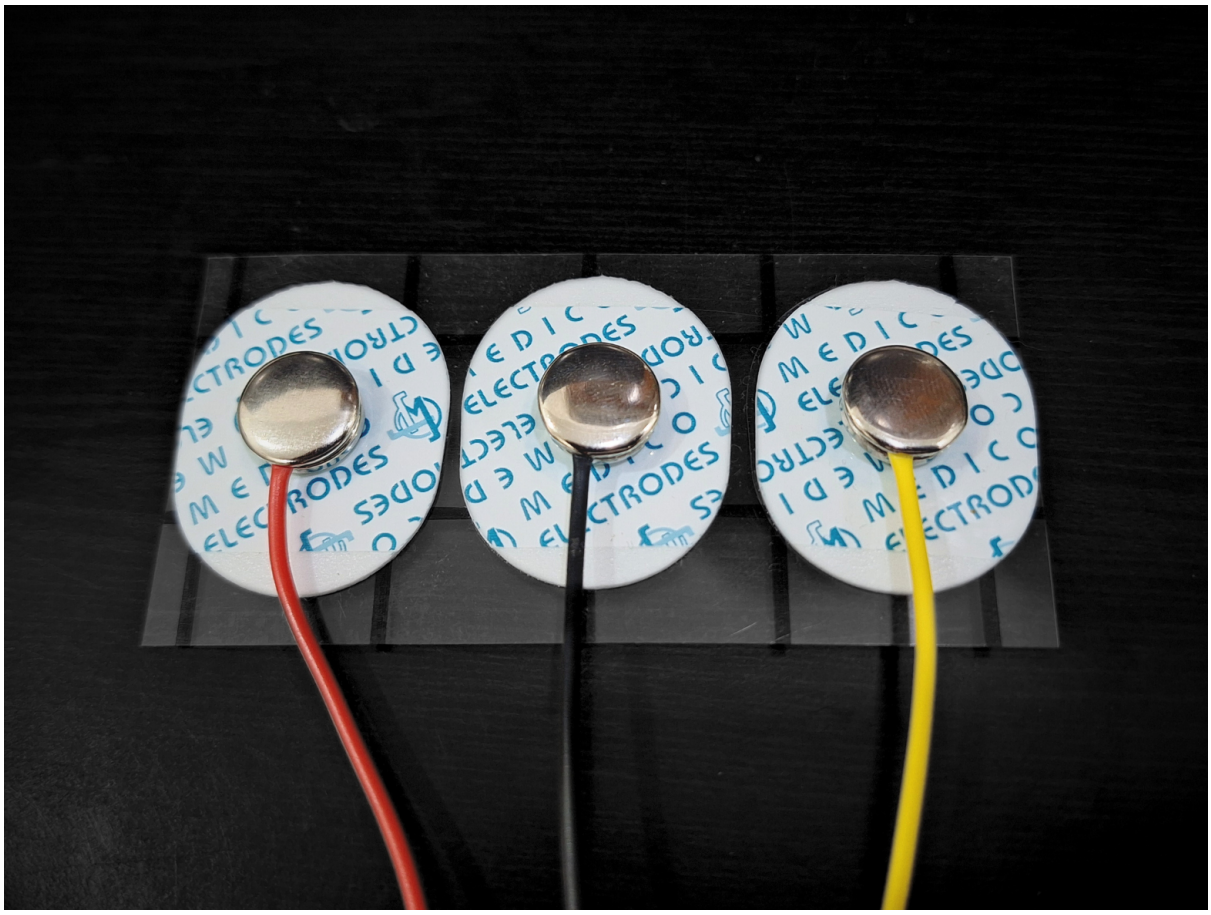

### 2.3.2 2. Classification on the basis of connectivity

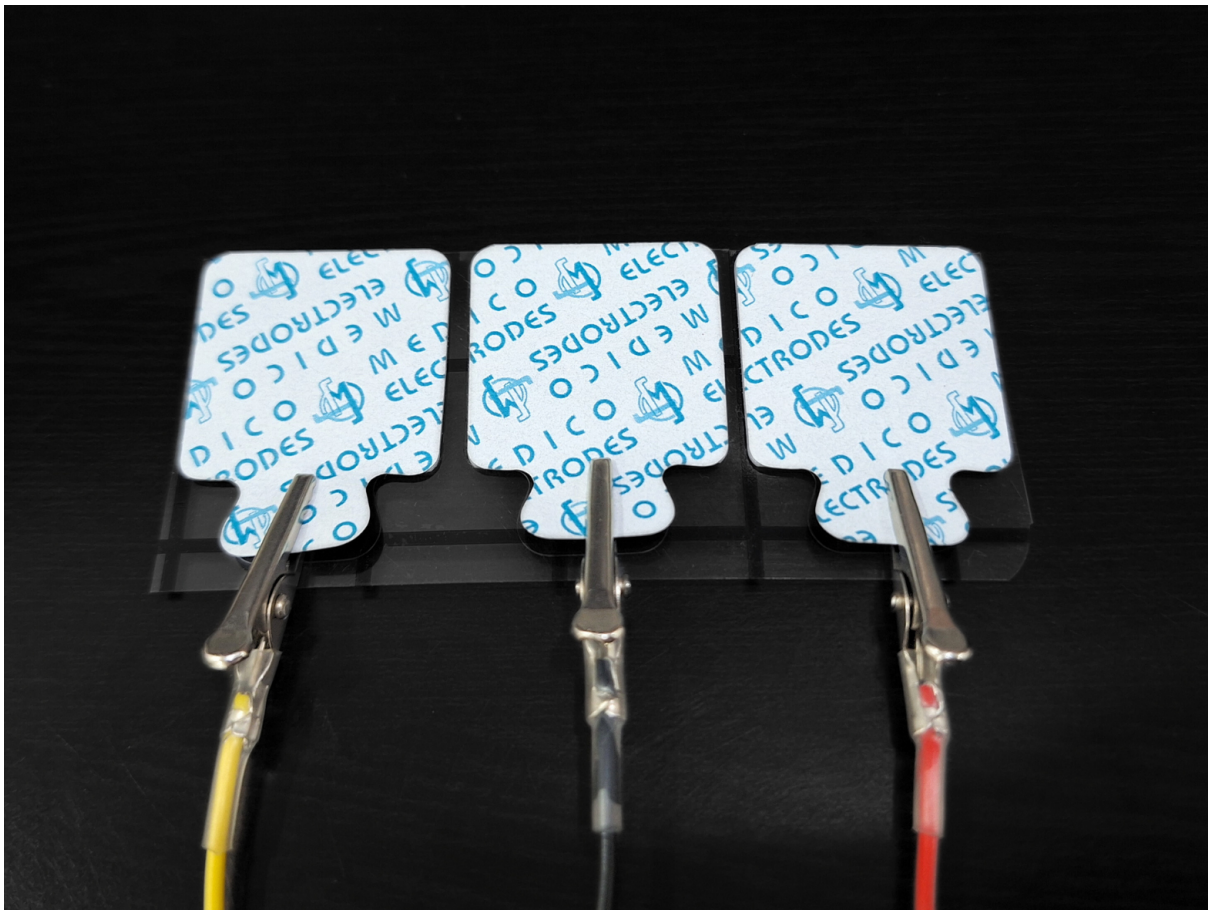
#### a. Connecting via snap

Some electrodes have a metallic snap connector on them which enables you to connect BioAmp Snap Cable by snapping the dry electrode part of the cable onto the electrode.

#### b. Connecting via tab

Some electrodes do not have the metallic snap connector on them, instead they have a conductive tab extending from the surface of the electrode. You can directly use BioAmp Alligator Cable to connect to the tab.





## 2.4 Using the electrodes

Determine the target area from where you want to record the biopotential signals.

**Warning**

For people having sensitive skin, it is recommended to use either gel electrodes with hydrogel/standard adhesive or use BioAmp Bands.

### 2.4.1 1. Skin Preparation

Remove any excessive hair on the targeted area. Apply Nuprep skin preparation gel on the skin surface where electrodes would be placed to remove dead skin cells and clean the skin from dirt. After rubbing the skin surface thoroughly, clean it with an alcohol wipe or a wet wipe.

For more information, please check out detailed step by step Skin Preparation Guide.

**Note**

Always ensure that the prepared skin area is dry prior to applying the gel electrodes.

### 2.4.2 2. Connecting the cable

Connect the BioAmp Snap/Alligator Cable on the gel electrodes.

### 2.4.3 3. Electrodes placement

Peel of the plastic backing and place the electrodes on your targeted skin surface as per the connections directed with every BioAmp hardware.

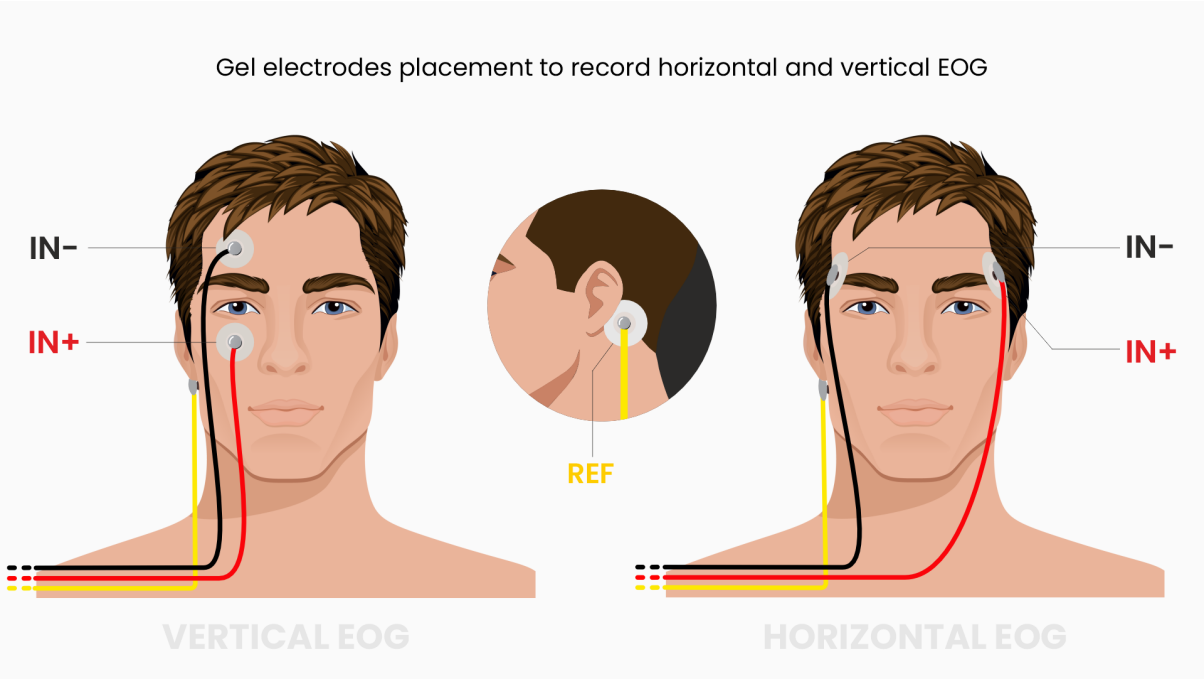



Fig. 1: Gel electrodes placement for EOG signal recording

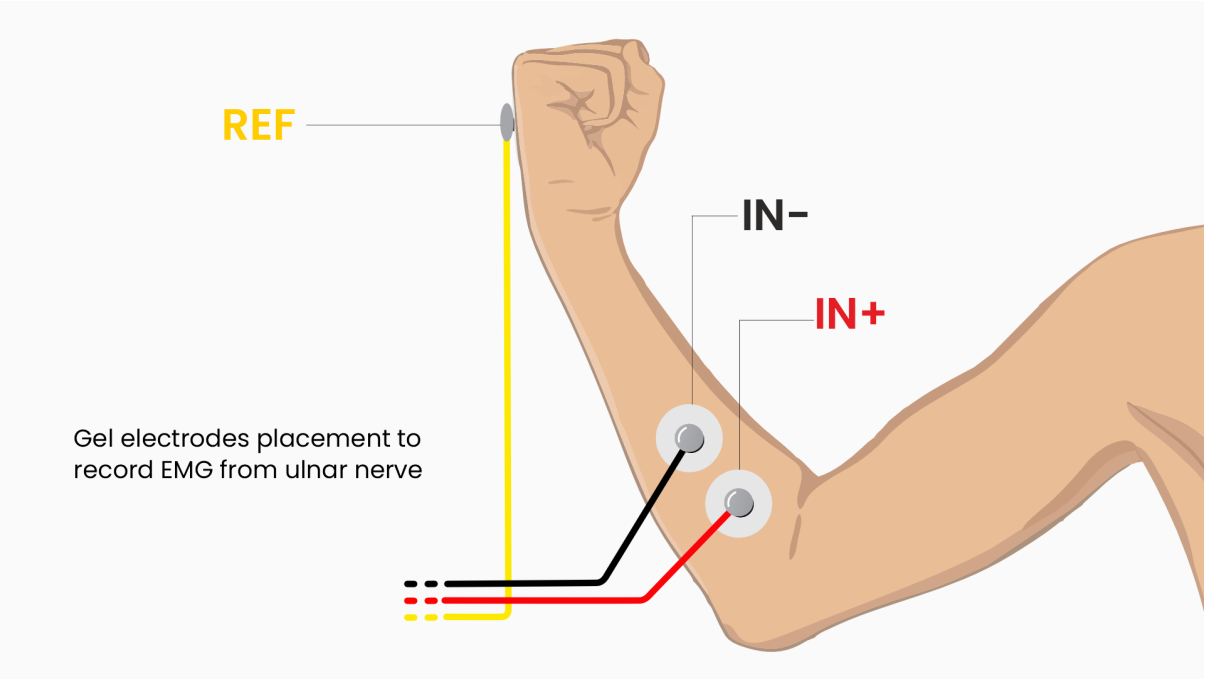



Fig. 2: Gel electrodes placement for EMG signal recording

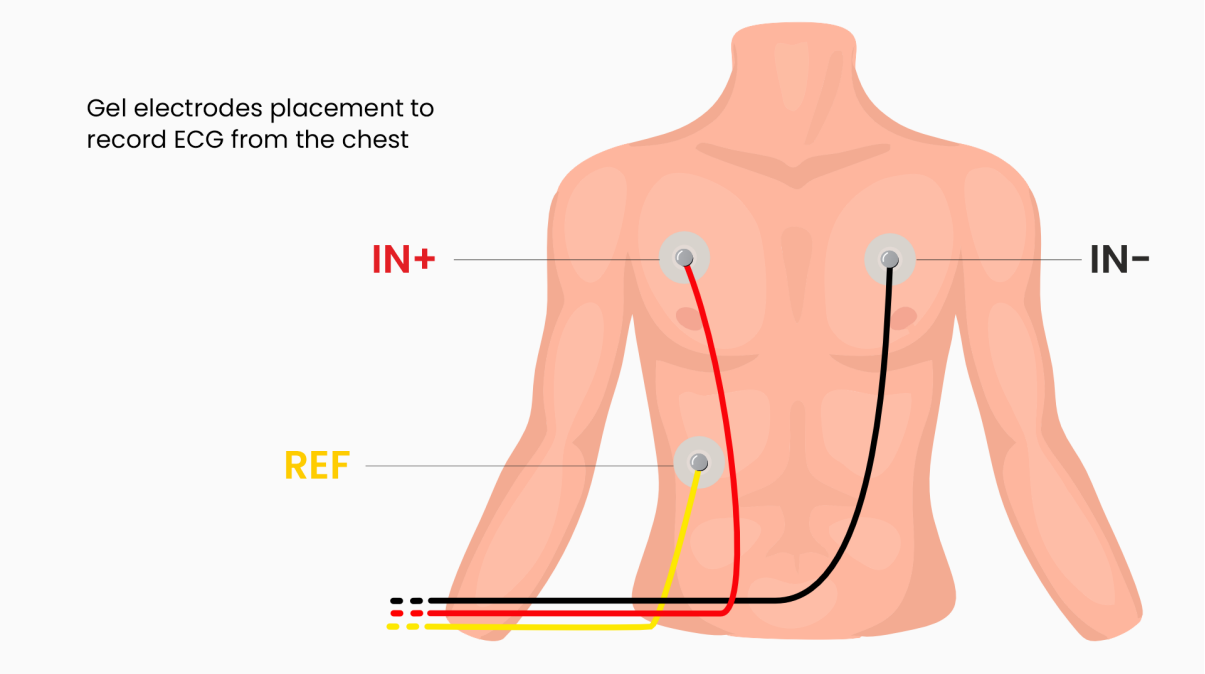



Fig. 3: Gel electrodes placement for ECG signal recording

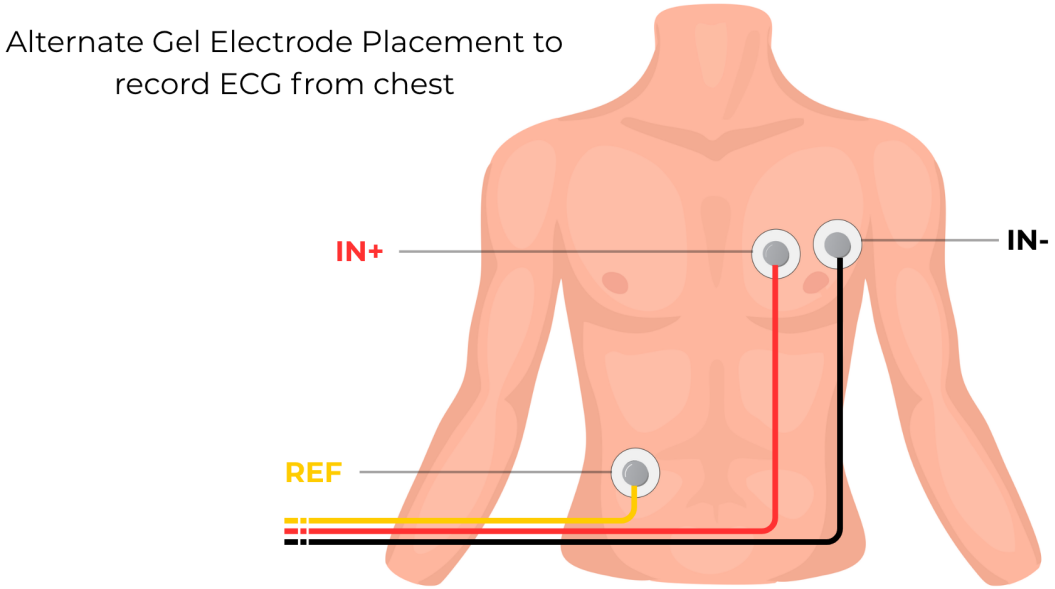



Fig. 4: Alternate Gel electrodes placement for ECG signal for applications like *Breathe Monitor*

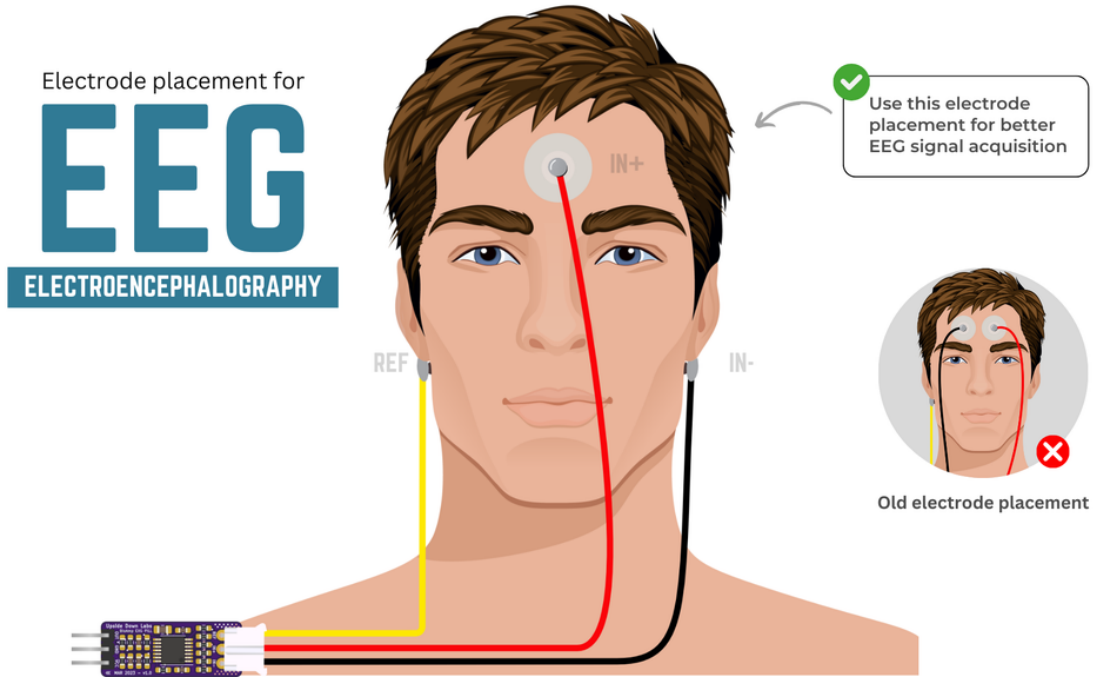



Fig. 5: Gel electrodes placement for EEG signal recording

**Tip**

While placing the gel electrodes on the skin, make sure to place the non-sticky tab of the electrode in the direction opposite to your hair growth. This allows you to remove the electrodes easily without pulling off much body hair.

## 2.5 Removing the electrodes

Once you are done with your signal recording, pull the non-sticky tab gently in the direction of your body hair growth to remove the gel electrode from the skin.

- To remove the gel residue with a wet wipe/alcohol swab.
- To remove the adhesive residue apply some oil to your skin and rub it gently for a couple of minutes to dissolve the adhesive, and then wipe it off with a dry cloth.

**Important**

You should not use water & soap to clean the adhesive residue, it is oil soluble and should be dissolved with oil before removing it with dry cloth. Using water and soap can make it stick harder and irritate your skin.

# 3 Chords-Web

https://youtu.be/vwmID4WjRrE?si=QdjEdg9xENhE_Ubp

## 3.1 Overview

Chords-Web is an open-source web application for real-time visualization of bio-potential signals like EEG, EMG, ECG and EOG. It is an advanced alternative to the standard Arduino IDE serial plotter, made for students, researchers and hobbyists who want to record biopotential data using BioAmp hardware.

Chords-Web runs completely in your browser - there is nothing to install. Open chords.upsidedownlabs.tech, connect your board over USB or Bluetooth, and start visualizing.

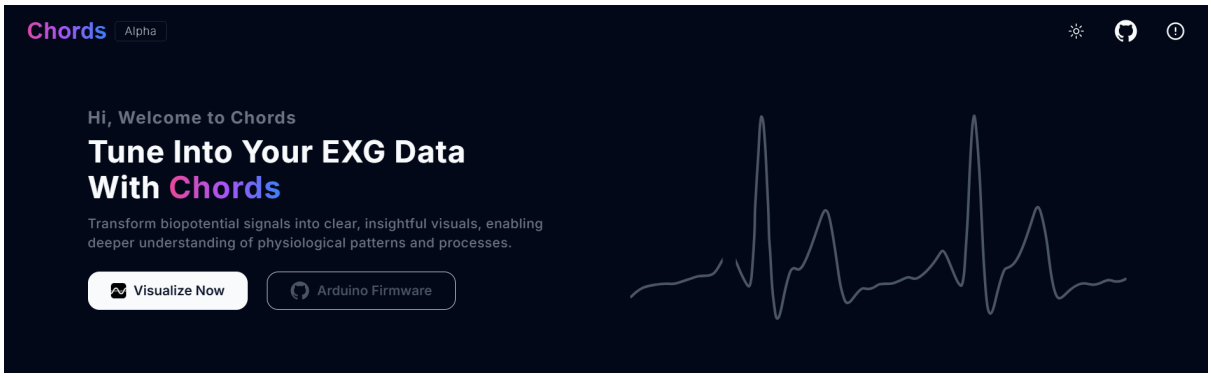



Fig. 1: Chords-Web Landing Page

**Note**

Chords-Web is made for **educational and research purposes**. It is not medical software and must not be used for diagnosis.

## 3.2 Quick Start

If you are new to Chords-Web, follow these five steps. Each step is explained in detail in the sections below.

1. **Hardware**: Connect your BioAmp sensor to a *supported development board* and place the electrodes on your body.
2. **Firmware**: Upload the Chords firmware to your board (see *Uploading the Firmware*).
3. **Open Chords-Web**: Go to chords.upsidedownlabs.tech in a *supported browser* and click **Visualize Now**.
4. **Connect**: Click **Serial** (USB cable) or **BLE** (Bluetooth, NPG Lite only) in the bottom bar and select your device.
5. **Choose a view**: Use the app switcher in the bottom bar to open the *Chords Visualizer*, *FFT Visualizer*, *Rep-Forge* or *ECG Visualizer*.

## 3.3 Requirements

### 3.3.1 Browser Compatibility

Chords-Web talks to your board directly from the browser using these web APIs:

- **Web Serial API**: to connect boards over a USB cable.
- **Web Bluetooth API**: to connect the NPG Lite over Bluetooth Low Energy (BLE).
- **WebUSB API**: to update the firmware of some boards from the browser.

Not every browser supports all three APIs, so some features may not work in your browser:

| Browser | Serial (USB) | Bluetooth (BLE) | Firmware update |
|---|---|---|---|
| Google Chrome, Microsoft Edge, Opera | Yes | Yes | Yes (all boards) |
| Mozilla Firefox 151 or later | Yes | No | Some boards only |
| Safari | No | No | No |

For all features, use the latest version of **Google Chrome**, **Microsoft Edge** or **Opera**. Firefox supports Web Serial from **version 151**. Older Firefox versions can't connect to your board and will show the unsupported browser message.

If your browser does not support the Web Serial API, Chords-Web shows a message telling you to switch to a supported browser.

**Warning**

**Using Firefox 151 or later?** You can connect your board over **Serial** and use all the applications. But Firefox doesn't support Web Bluetooth or WebUSB, so:

- The **BLE** button won't work, so you can't connect the NPG Lite over Bluetooth. Connect it over Serial instead.
- The in-browser *firmware update* doesn't work for boards that are flashed over WebUSB: Arduino UNO R4 Minima, Arduino GIGA R1, STM32G4 Core Board, STM32F4 Black Pill and Raspberry Pi Pico. Update these boards using Chrome/Edge or the Arduino IDE. Other boards can be updated in Firefox, because they are flashed over Web Serial.

**Important**

Chords-Web works only on **desktop/laptop** computers. If you open it on a phone, you will see an "Oops! Mobile Detected" message.

**Tip**

**Brave** is Chromium-based, but it turns off Web Serial and Web Bluetooth by default. If you use Brave, enable them from `brave://settings/content` or use Chrome/Edge instead.

For more information, refer to MDN Web Docs on the Web Serial API and the Web Bluetooth API.

### 3.3.2 Software Requirements

- A *supported browser*.
- Chords firmware on your board. You can upload it in any of these ways:
  - Directly from Chords-Web (see *Option 1: Update firmware from Chords-Web (Recommended)*).
  - Using the Arduino IDE and the Chords Arduino Firmware.
  - Using the NPG Lite Flasher Web (in your browser) or the NPG Lite Flasher (desktop app) for the NPG Lite or other ESP32 based boards.

### 3.3.3 Hardware Requirements

- A *supported development board* (or an NPG Lite, which has a built-in BioAmp).
- A USB cable (type depends on your board).
- BioAmp Hardware and its accessories (cables, electrodes, gel).

### 3.3.4 Supported Boards

Chords-Web detects your board automatically and sets the ADC resolution, number of channels and sampling rate for you.

| Board | ADC Resolution | Channels | Sampling Rate | Connection |
|---|---|---|---|---|
| Arduino UNO R3 / Genuino UNO / Maker UNO (UNO clone) | 10-bit | 6 | 250 Hz | Serial |
| Arduino Nano Classic / Nano clone | 10-bit | 8 | 250 Hz | Serial |
| Arduino MEGA 2560 R3 / MEGA 2560 clone | 10-bit | 16 | 250 Hz | Serial |
| Arduino UNO R4 Minima / UNO R4 WiFi | 14-bit | 6 | 500 Hz | Serial |
| Arduino GIGA R1 | 16-bit | 6 | 500 Hz | Serial |
| Raspberry Pi Pico (RP2040) | 12-bit | 3 | 500 Hz | Serial |
| STM32G4 Core Board | 12-bit | 16 | 250 Hz | Serial |
| STM32F4 Black Pill (STM32F401CC / STM32F411CE) | 12-bit | 8 | 500 Hz | Serial |
| ESP32-S3 | 12-bit | 16 | 250 Hz | Serial |
| NPG Lite (3 channel / 6 channel) | 12-bit | 3 or 6 | 500 Hz | Serial or BLE |

**Tip**

You can also use any other microcontroller, as long as it sends data in the *Chords data format*.

## 3.4 Setting up the Hardware

Make all the connections for the hardware you are using:

1. Connect the BioAmp sensor to the analog input pins of your development board.
2. Connect the electrodes from the BioAmp sensor to your body.
3. Connect the development board to your laptop using a USB cable.

You can see diagrams of these connections (cable connections, BioAmp to board connections and electrode placement for ECG and EOG) on the Chords-Web page after you click **Visualize Now**, before you connect a device.

## 3.5 Uploading the Firmware

Your board needs the Chords firmware so that it can send data to Chords-Web. Choose any one of the options below.

### 3.5.1 Option 1: Update firmware from Chords-Web (Recommended)

You don't need the Arduino IDE for this option.

1. Connect your board to your laptop with a USB cable.
2. Open chords.upsidedownlabs.tech, click **Visualize Now** and then click **Serial**.
3. Select your board's port and click **Connect**.
4. If the board is supported but doesn't have the latest Chords firmware, a **"Your firmware is not updated"** dialog opens.
5. If several boards share the same USB ID (for example, UNO clones and Nano clones), select which board you have.
6. Click **Flash firmware**. Chords-Web downloads the latest firmware release and uploads it to your board.
7. When it shows **Firmware updated**, Chords-Web reconnects to your board automatically.

**Note**

- **STM32 boards**: If the board is still in normal mode, put it in boot mode first. Hold **BOOT0**, tap **NRST**, release **BOOT0**, and click **Flash firmware** again.
- **Windows**: Some boards need a USB driver before they can be flashed (only once per PC). If flashing doesn't start within 15 seconds, follow the steps in the dialog to install the driver using **ChordsDriverSetup.exe** or **Zadig** (choose **WinUSB** and click **Install Driver**). Then replug the board and click **Flash firmware** again.

### 3.5.2 Option 2: Upload firmware with the Arduino IDE

1. Connect your board to your laptop with a USB cable.
2. Go to the Chords Arduino Firmware GitHub repository, scroll to the supported boards table and open the sketch for your board.
3. Copy the sketch and paste it into the Arduino IDE.
4. Go to `Tools` > `Board` and select your board.
5. Go to `Tools` > `Port` and select your board's port. This is the port that disappears when you unplug the board:

- **Windows**: `COM3`, `COM4`, etc.
- **Linux**: `/dev/ttyACM0` or `/dev/ttyUSB0`
- **macOS**: `/dev/cu.usbmodem...` or `/dev/cu.usbserial...`

6. Click the **Upload** button.
7. Close the Arduino IDE Serial Monitor/Plotter (only one app can use the port at a time) and open Chords-Web.

### 3.5.3 Option 3: NPG Lite and ESP32 boards

Use one of these tools to upload the Serial or BLE firmware to your NPG Lite or any ESP32 based board:

- **NPG Lite Flasher Web**: Runs in your browser, nothing to install. Open upsidedownlabs.github.io/NPG-Lite-Flasher-Web, click **Connect Device**, select the **BLE** or **Serial** firmware and click **Flash Firmware**.
- **NPG Lite Flasher**: A desktop app for Windows, macOS and Linux.

## 3.6 Connecting to Chords-Web

1. Open a *supported browser* and go to chords.upsidedownlabs.tech.
2. Click **Visualize Now**.
3. The images in the center show how to connect your hardware.
4. The bottom bar has three buttons:
   - **BLE**: Connect an NPG Lite over Bluetooth.
   - **Serial**: Connect any supported board over a USB cable.
   - **Serial Wizard**: Open the *Serial Wizard Plotter & Monitor*.

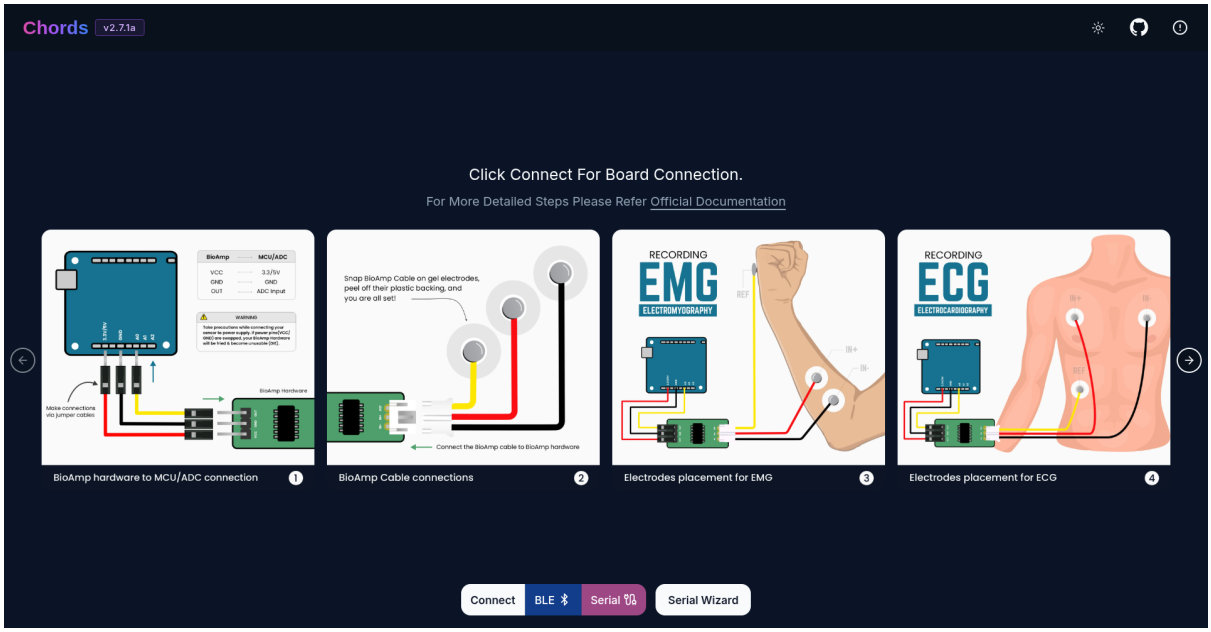



Fig. 2: Chords-Web Connect Page

### 3.6.1 Connecting over Serial (USB)

1. Click **Serial**.
2. Your browser shows a list of serial ports. Select your board and click **Connect**.
3. Chords-Web detects your board, and the Chords Visualizer starts plotting your signals.

#### Linux: Board not listed or "permission denied"

On Linux your user must be allowed to access serial ports. Add yourself to the right group, then log out and log back in:

- **Arch / Manjaro**:

```
sudo usermod -aG uucp $USER
```

- **Ubuntu / Debian**:

```
sudo usermod -aG dialout $USER
```

### 3.6.2 Connecting over Bluetooth (NPG Lite)

The NPG Lite can stream data wirelessly over Bluetooth Low Energy (BLE). It has a built-in BioAmp and a 12-bit ADC (ESP32-C6), so you don't need any other development board.

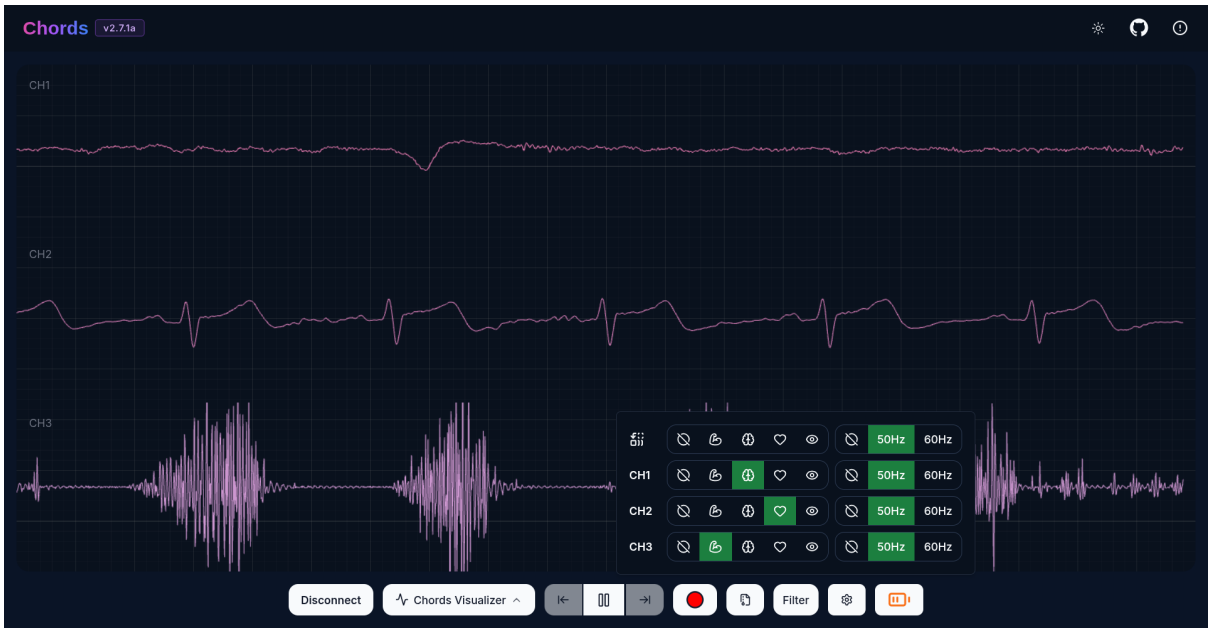



Fig. 3: Chords-Web NPG Lite

1. **Flash the BLE firmware**: Use the NPG Lite Flasher Web or the NPG Lite Flasher to upload the **BLE firmware** (3 channel or 6 channel) to your NPG Lite.
2. **Power on** the NPG Lite using its on/off switch.
3. **Turn on Bluetooth** on your computer.
4. Open Chords-Web, click **Visualize Now** and then click **BLE**.
5. Select your NPG Lite (its name starts with `NPG`) from the list and click **Pair**.
6. Your signals start plotting. You can now use any of the *applications* below.

**Features over BLE:**

| Feature | Description |
|---|---|
| **Wireless Streaming** | Stream 3 or 6 channels of biopotential data at 500 Hz without a cable. |
| **Battery Indicator** | The bottom bar shows the battery level of your NPG Lite. You get a warning when the battery is low and a message when it is fully charged (3CH/6CH firmware only). |
| **Interactive Controls** | Select channels, play/pause the stream, apply filters and record to CSV, just like over Serial. |

**Note**

If you don't see the battery level, your NPG Lite has older BLE firmware. Update it with the NPG Lite Flasher Web or the NPG Lite Flasher.

**Note**

Checkout our YouTube video for more information:

https://youtu.be/3YCioyc4uKs

## 3.7 Common Controls

These controls are in the bottom bar and work in all the applications, unless mentioned otherwise.

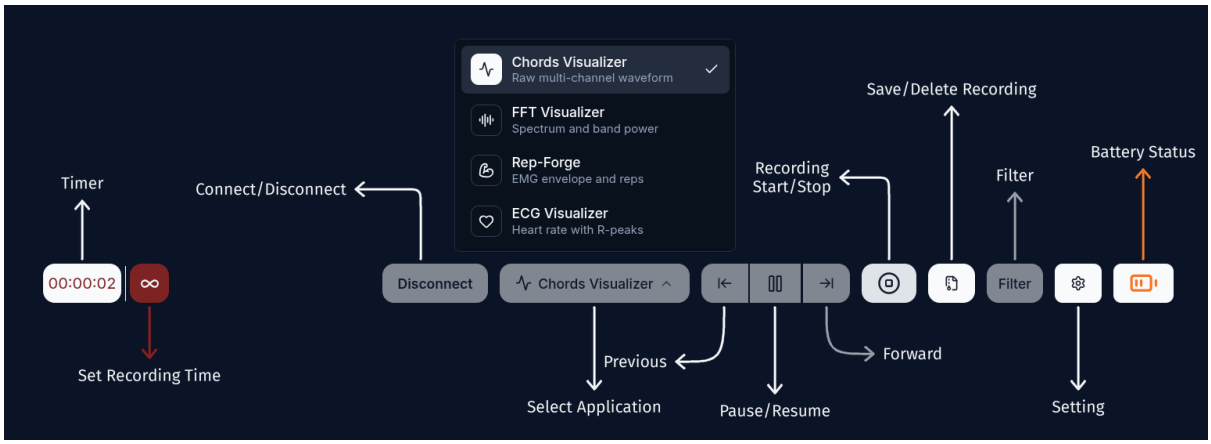



Fig. 4: Chords-Web Icons

### 3.7.1 App Switcher

After you connect, use the app switcher in the bottom bar to move between the four applications. You don't need to reconnect your board when you switch.

- **Chords Visualizer**: Raw multi-channel waveform.
- **FFT Visualizer**: Spectrum and EEG band power.
- **Rep-Forge**: EMG envelope and muscle strength.
- **ECG Visualizer**: Heart rate with R-peaks.

**Note**

Filters are reset every time you switch to another application, so a filter you chose in one view doesn't keep running in another.

### 3.7.2 Play/Pause and Data Windows

- Click **Pause** to freeze the screen on the current data window.
- While paused, use the **Previous Data Window** (left) and **Next Data Window** (right) buttons to go back through up to **5 earlier data windows**.
- Click **Play** to continue the live stream.
- Changing the selected channels resets the saved data windows.
- Pause is not available while recording.

### 3.7.3 Settings (Zoom, Time Base and Channels)

Click the **Settings** icon to open the settings popover:

- **Zoom Level**: Zoom from **1x to 10x** to see small signal details or the full signal.
- **Time Base**: Show **1 to 10 seconds** of data on the screen.
- **Channels**: Select the channels you want to plot. The number of available channels depends on your board (up to 16). Use **Select All** to choose all channels and **Reset** to go back to your previous selection.

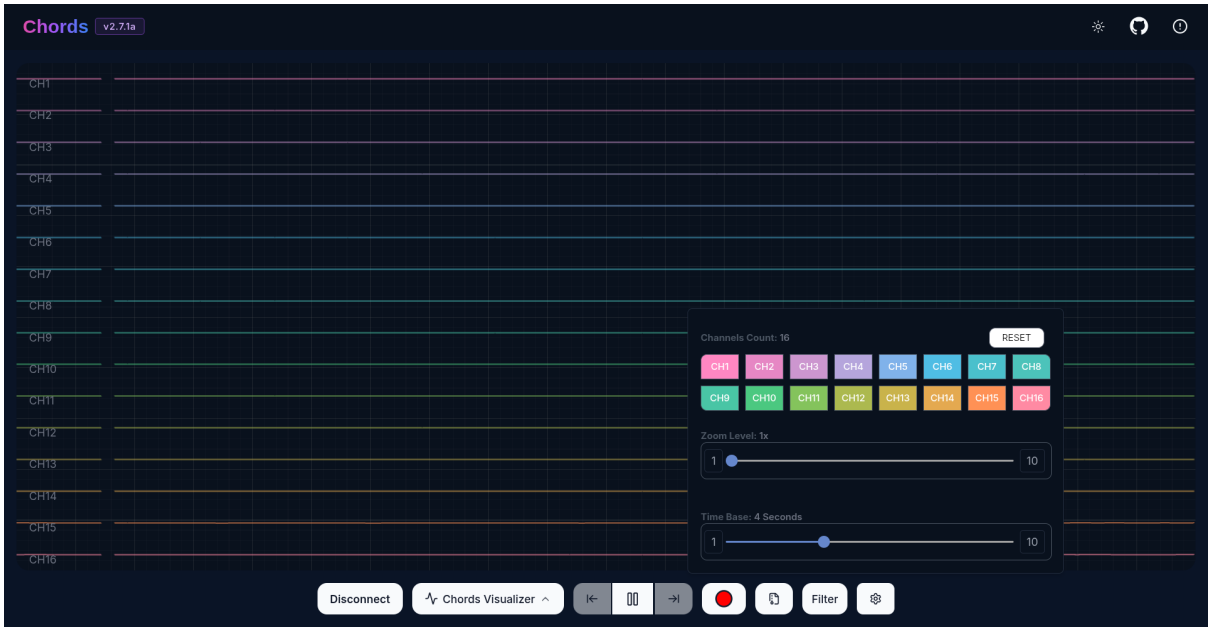



Fig. 5: Chords-Web Channel Support

### 3.7.4 Filters

Click **Filter** to clean up your signal. You can apply a filter to one channel or to all channels at once using the master buttons at the top.

| Filter | Type | Use it for |
|---|---|---|
| **Muscle** | 70 Hz high-pass | EMG (muscle) signals |
| **Heart** | 30 Hz low-pass | ECG (heart) signals |
| **Eye** | 10 Hz low-pass | EOG (eye) signals |
| **Brain** | 45 Hz low-pass | EEG (brain) signals |
| **50 Hz / 60 Hz** | Notch | Removes power line noise. Use 50 Hz in India, Europe, etc. and 60 Hz in the USA, etc. |

Fig. 6: Chords-Web Filters

**Note**

- A **1 Hz high-pass** filter is always applied to remove DC drift, even when no other filter is selected.
- The Muscle, Heart, Eye and Brain filters are available in the **Chords Visualizer**. **Rep-Forge** has only the Muscle filter and the 50/60 Hz notch filter.

### 3.7.5 Recording the Data

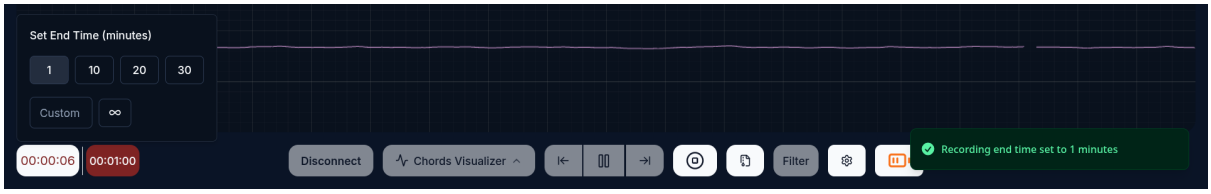



Fig. 7: Recording Time

1. Click the **Record** (red circle) button to start recording in **CSV format**.
2. To stop automatically, set an end time: choose **1, 10, 20 or 30 minutes**, or type your own time in minutes. You can also record without a time limit and click **Stop** when you are done.
3. Recordings can be at most **24 hours** (1440 minutes) long.

Recorded files are saved in your browser (using **IndexedDB**), so they are not lost if you disconnect your board. Click the **Files** button to manage them:

- **Download** or **delete** a single file.
- **Download All as Zip** or **Delete All** with one click.

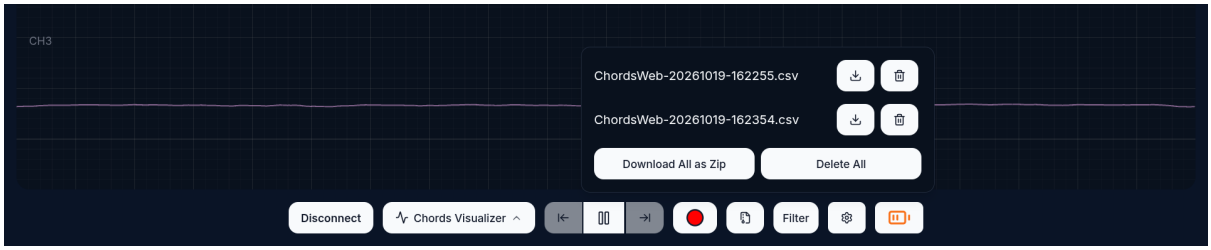



Fig. 8: Save and Delete Option

Each CSV file has a `Counter` column followed by one column per selected channel (`Channel1`, `Channel2`, ...). You can plot these files with the Chords CSV Plotter.

### 3.7.6 Disconnecting

Click the **Disconnect** button to stop the data stream and disconnect your board.

### 3.7.7 Other Options to Explore

- **Switch Theme**: Switch between light and dark mode using the theme button in the navigation bar.
- **Version**: The navigation bar shows the version of Chords-Web you are using.
- **GitHub Repository**: Open the Chords-Web GitHub repository using the link in the navigation bar.
- **Contributors**: See the list of contributors using the link in the top-right corner of the navigation bar.

## 3.8 Applications

### 3.8.1 Chords Visualizer

The Chords Visualizer is the default application. It plots your raw biopotential signals from multiple channels in real time. Use it to monitor EMG, ECG, EOG or EEG signals, apply filters and record your data.

#### Features

| Feature | Description |
|---|---|
| **Auto Board Detection** | Instantly detects **BioAmp Hardware** running the **Chords firmware** and sets the ADC resolution, channels and sampling rate automatically. |
| **Real-time Visualization** | Smooth, real-time data rendering powered by WebGL-Plot. |
| **Multi-channel Plotting** | Plots **up to 16 channels**, depending on your board, each in a different color. |
| **Data Windows** | Pause and go back through the **last 5 data windows**. |
| **Zoom & Time Base** | Zoom from 1x to 10x and show 1 to 10 seconds of data on the screen. |
| **Filters** | Muscle, Heart, Eye and Brain filters, and 50/60 Hz notch filters, per channel or for all channels. |
| **Recording & Data Management** | Record data in **CSV format** with or without a timer. Download or delete files individually or as a ZIP. |

### Running the Application

1. Connect your board (see *Connecting to Chords-Web*). The Chords Visualizer opens automatically.
2. Open **Settings** to select channels and adjust the zoom and time base.
3. Click **Filter** and choose a filter for the signal you are recording (for example, **Muscle** for EMG).
4. Click **Record** to save your data, and use the **Files** button to download it.

### Visualizing EMG (Electromyography) signals

EMG captures the electrical activity produced by skeletal muscles. Use the **Muscle** filter.

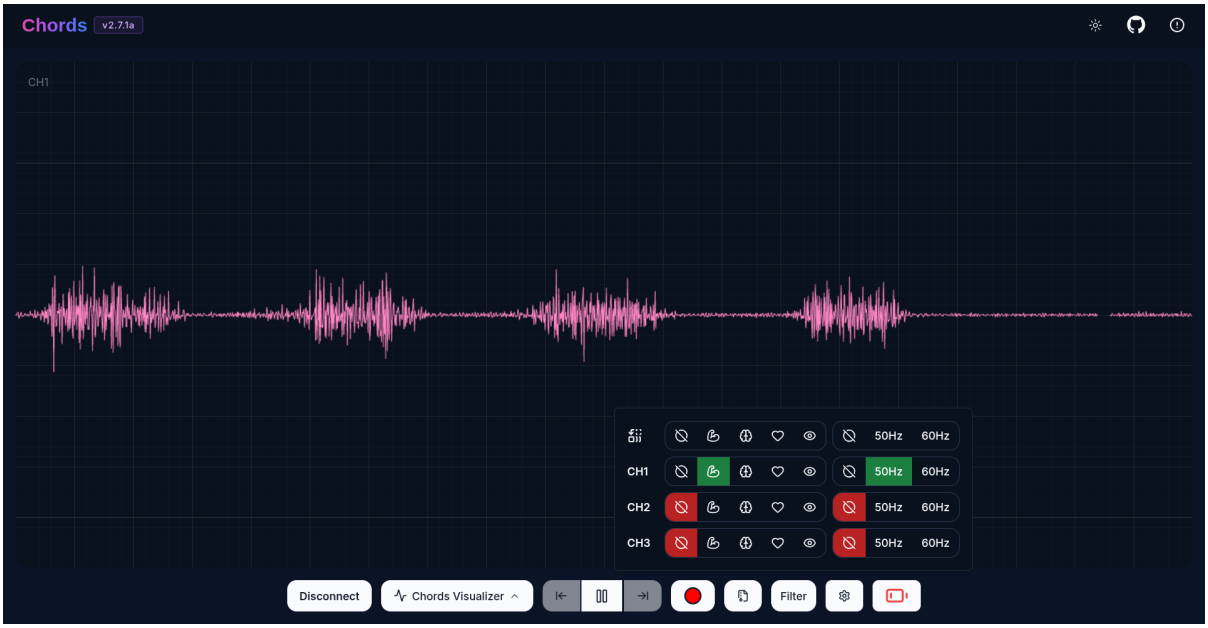



Fig. 9: EMG Signal Example

### Visualizing EEG (Electroencephalography) signals

EEG records the electrical activity of the brain. Use the **Brain** filter.

**Tip**

To ensure you're recording a high-quality signal, refer to the detailed guide here: Troubleshooting EEG Signal Quality.

### Visualizing EOG (Electrooculography) signals

EOG measures the electrical potential generated by eye movements. Use the **Eye** filter.

Fig. 10: EEG Signal Example

Fig. 11: EOG Signal Example

### Visualizing ECG (Electrocardiography) signals

ECG records the electrical activity of the heart and shows your heart rhythm. Use the **Heart** filter.

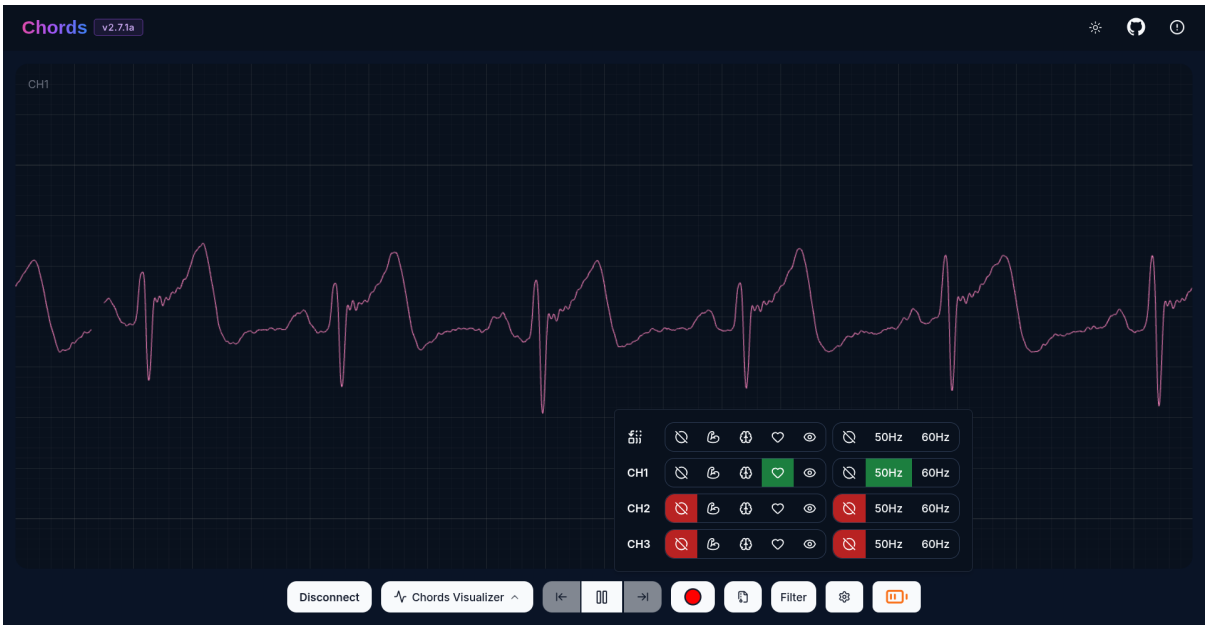



Fig. 12: ECG Signal Example

## 3.8.2 FFT Visualizer

### Overview

The FFT Visualizer uses **FFT (Fast Fourier Transform)** to show the frequencies present in your signal and the power of each **EEG band**. Use it to see your brainwaves and track your focus in real time.

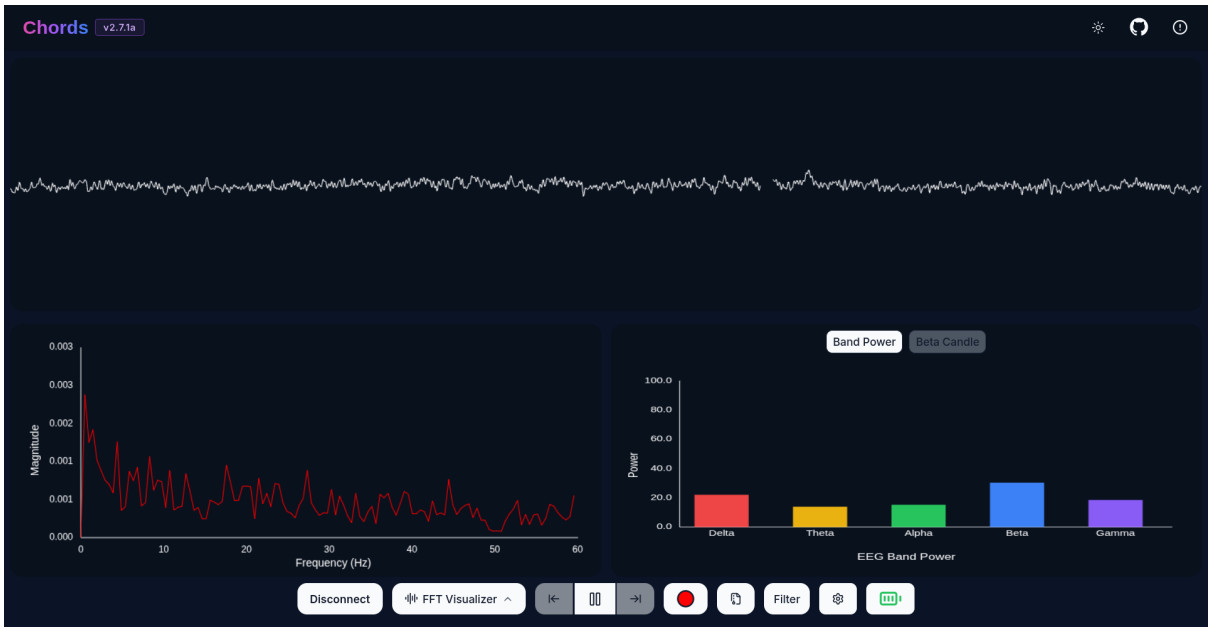



Fig. 13: Chords-Web FFT Visualiser

### Features

| Feature | Description |
|---|---|
| **Live EEG Band Monitoring** | View real-time EEG band power values to track brain activity. |
| **Beta Candle** | A candle that glows brighter when your beta (focus) activity is higher. |
| **Channel Selection** | Choose which channel to analyze from the **Settings** icon (Channel 1 by default). |
| **Data Windows** | Pause and go back through the last 5 data windows. |
| **Recording** | Record the data as a **CSV file** for further analysis. |

### Supported EEG Bands

- **Delta (0.5 - 4 Hz)** → Associated with deep sleep and unconscious states.
- **Theta (4 - 8 Hz)** → Linked to relaxation, meditation, and light sleep.
- **Alpha (8 - 13 Hz)** → Reflects calm, wakeful relaxation, often seen during closed-eye rest.
- **Beta (13 - 30 Hz)** → Related to active thinking, problem-solving, and focus.
- **Gamma (30 - 45 Hz)** → Involved in high-level cognitive functioning, attention, and perception.

### Running the Application

1. Connect your board over **Serial** or **BLE** (see *Connecting to Chords-Web*).
2. Select **FFT Visualizer** from the app switcher.
3. Open **Settings** and select the channel connected to your EEG electrodes.
4. The **top section** shows your EEG signal, filtered with the **Brain** filter (45 Hz low-pass) to remove noise.
5. The **bottom section** is divided into two parts:
    - **Left side** → The frequency spectrum (FFT) of your signal in Hz.
    - **Right side** → Two modes:
        - **Band Power Mode** → Real-time power of each EEG band.
        - **Beta Candle Mode** → A glowing candle that shows your focus level.
            - **Brighter candle** = Higher beta waves = Strong focus.
            - **Dim candle** = Lower beta waves = Distraction.

**Note**

Checkout our YouTube video for more information:

https://youtu.be/zkPGzX3GKnk

### 3.8.3 Rep-Forge

#### Overview

Rep-Forge shows the strength of your muscle contractions (EMG) in real time. Use it to monitor your muscles while you work out or during rehabilitation, directly in your browser.

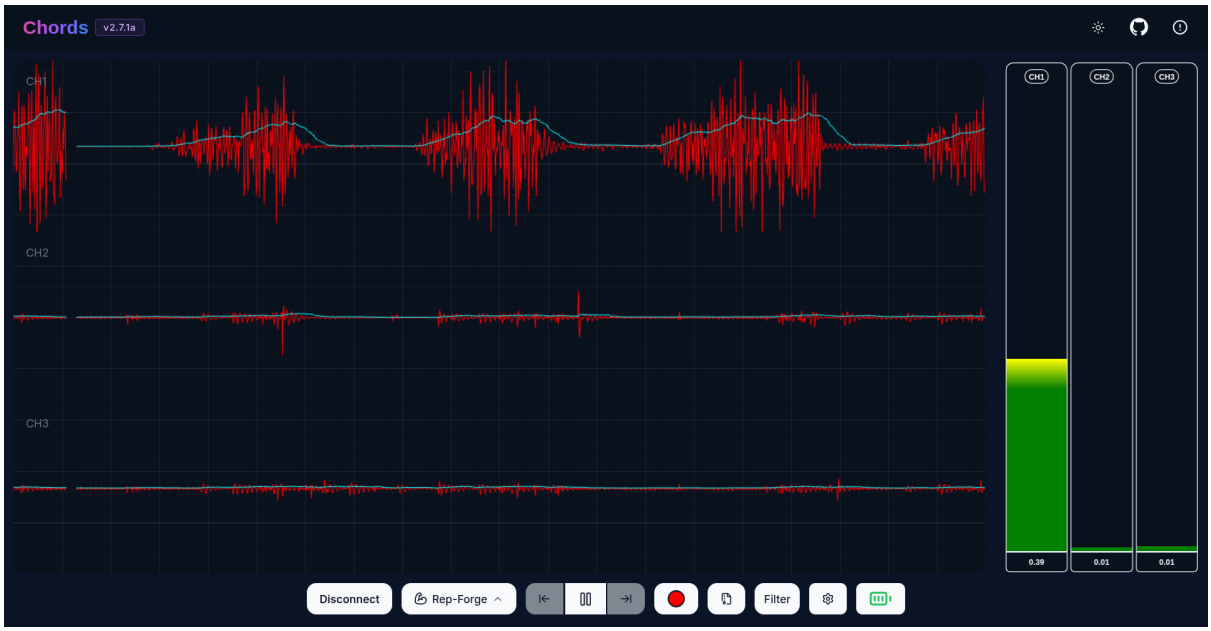



Fig. 14: Chords-Web Rep-Forge

On the left, each channel shows the raw EMG signal (red) with its envelope (blue) on top. On the right, the strength bar of each channel rises as the muscle contracts. In the image above, only CH1 is active.

#### Features

| Feature | Description |
|---|---|
| **Multi-channel EMG** | Monitor **up to 6 EMG channels** at the same time. |
| **EMG Envelope** | Shows the envelope of your EMG signal, which follows how strongly your muscle is contracting. |
| **Live Strength Bars** | A bar for each channel shows the contraction level in real time, with the channel name at the top and the current value at the bottom. |
| **Filters** | Muscle filter and 50/60 Hz notch filter to remove noise. |
| **Data Windows & Recording** | Pause and go back through the last 5 data windows, and record to CSV. |
| **Serial or BLE** | Works with any supported board over Serial, or with the NPG Lite over BLE. |

### Running the Application

1. Place the electrodes on the muscle you want to monitor and connect them to your BioAmp sensor.
2. Connect your board over **Serial** or **BLE** (see *Connecting to Chords-Web*).
3. Select **Rep-Forge** from the app switcher.
4. Open **Settings** and select up to 6 channels.
5. Contract your muscle and watch the strength bar for that channel go up.

### Use Cases

- **Strength Training**: Quantify muscle activation during lifts or repetitions.
- **Rehabilitation**: Monitor recovery progress in injured muscle groups.
- **Research**: Capture EMG data for neuromuscular studies.

## 3.8.4 ECG Visualizer

### Overview

The ECG Visualizer shows your **heart rate** in real time. It detects the **R-peaks** (the tallest spike of each heartbeat) in your ECG signal and calculates your heart rate in beats per minute (BPM).

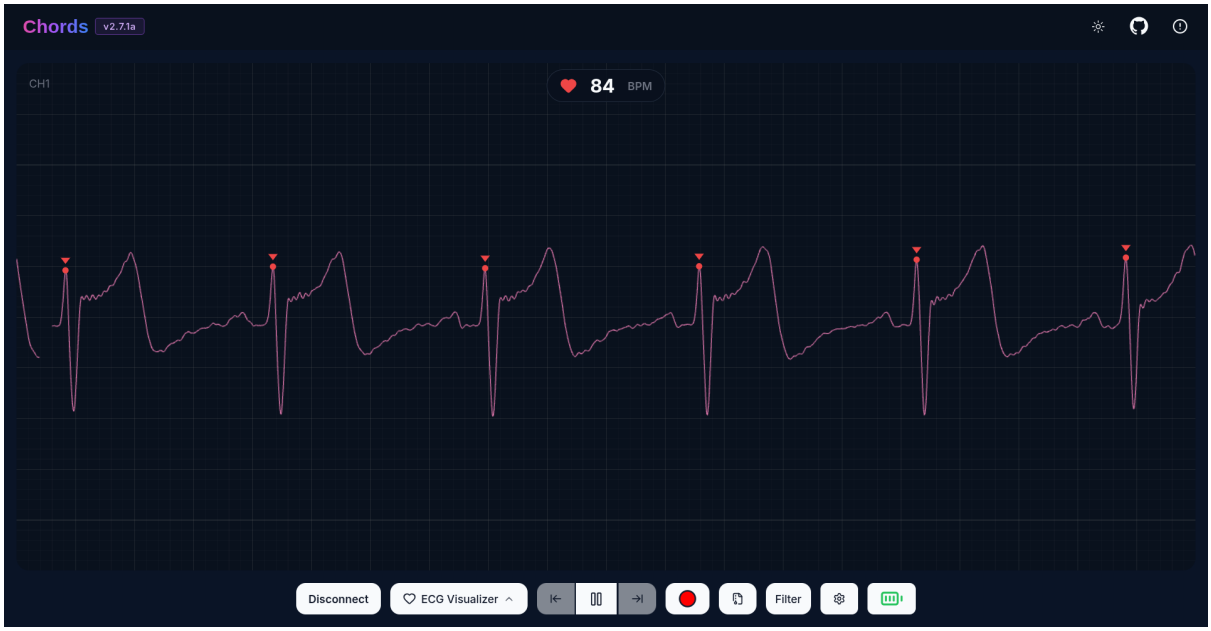



Fig. 15: Chords-Web ECG Visualizer

Each detected R-peak is marked with a red dot and arrow, and your heart rate is shown in BPM at the top.

### Features

| Feature | Description |
|---|---|
| **Heart Rate (BPM)** | Shows your live heart rate, with a heart icon that beats with each detected heartbeat. |
| **R-peak Detection** | Detects R-peaks using the Pan-Tompkins algorithm. |
| **Automatic ECG Filter** | The **Heart** filter is turned on for all channels automatically, for a clean ECG trace. |
| **Channel Selection** | Choose which channel to analyze from the **Settings** icon (Channel 1 by default). |

### Running the Application

1. Place the ECG electrodes on your body (the electrode placement diagram is shown on the Chords-Web connect page).
2. Connect your board over **Serial** or **BLE** (see *Connecting to Chords-Web*).
3. Select **ECG Visualizer** from the app switcher.
4. Open **Settings** and select the channel connected to your ECG sensor.
5. Sit still and relaxed. After a few heartbeats, your heart rate appears on the screen.

**Note**

The ECG Visualizer is for learning and experiments only. It is not medical software.

## 3.8.5 Serial Wizard Plotter & Monitor

### Overview

The **Serial Wizard Plotter & Monitor** is a standalone tool in Chords-Web that plots and prints data from any serial device, similar to the Arduino IDE Serial Plotter and Serial Monitor. Unlike the other applications, it works with any sketch - it doesn't need the Chords firmware.

Fig. 16: Chords-Web Serial Wizard

#### Features

| Feature | Description |
|---|---|
| **Dual View Modes** | Switch between the **Plotter**, the **Monitor**, or both together. |
| **Optimized Data Rendering** | Handles high-speed data without cluttering the display, so your plot stays clear and accurate. |
| **Footer Button Bar** | Switch between viewing modes using the buttons at the bottom. |
| **Baud Rate Selection** | Choose from 9600, 19200, 38400, 57600, 115200 (default), 230400, 460800 or 921600 baud. |
| **Navigation Bar** | Switch theme (light/dark), visit the GitHub repository, view contributors or go back to the previous page. |

#### Running the Application

1. Open Chords-Web, click **Visualize Now** and then click **Serial Wizard**.
2. Select the **Baud Rate**. It must match the value in your sketch's `Serial.begin()`.
3. Click **Connect** and select your board's port.
4. Use the footer buttons to switch between the Plotter, the Monitor or both.

**Note**

Checkout our YouTube video for more information:

https://youtu.be/-C_QUpwcEJQ

## 3.9 Data Format and Data Quality

### 3.9.1 Data Format

This section is for you if you want to use Chords-Web with your own board or firmware. The Chords Arduino Firmware already does all of this, so use it as a reference.

#### Commands

Chords-Web sends these text commands to the board over Serial. Each command ends with a newline (`\n`):

| Command | What the board must do |
|---|---|
| `WHORU` | Reply with its board ID on one line, for example `UNO-R3`. Chords-Web uses this ID to set the channel count, ADC resolution and sampling rate. |
| `START` | Start sending data packets. |
| `STOP` | Stop sending data packets. |

The board ID must be one of the IDs Chords-Web knows: `UNO-R3`, `GENUINO-UNO`, `UNO-CLONE`, `NANO-CLASSIC`, `NANO-CLONE`, `MEGA-2560-R3`, `MEGA-2560-CLONE`, `UNO-R4`, `GIGA-R1`, `RPI-PICO-RP2040`, `STM32G4-CORE-BOARD`, `STM32F4-BLACK-PILL`, `NPG-LITE`, `NPG-LITE-3CH`, `NPG-LITE-6CH` or `ESP32-S3`. If you use your own board, reply with the ID of a supported board that has the same channel count, ADC resolution and sampling rate (see *Supported Boards*).

Chords-Web opens the port at **230400 baud**, unless it recognizes the board's USB ID (for example, Arduino UNO R3, Nano and MEGA boards use 115200 baud).

#### Packet Format

After `START`, the board sends one binary packet for every sample:

```
| 0xC7 | 0x7C | counter | CH1 high | CH1 low | ... | CHn high | CHn low | 0x01 |
```

- `0xC7 0x7C`: two sync bytes that mark the start of a packet.
- `counter`: one byte (`uint8_t`) from 0 to 255. It goes up by 1 with every packet and starts again from 0 after 255.
- `CH high, CH low`: the raw ADC value of each channel, as two bytes in big-endian order (high byte first).
- `0x01`: the end byte.

So each packet is `2 × channels + 4` bytes long. For example, a 6-channel board sends 16-byte packets. The board must always send all of its channels, even if only some are selected in Chords-Web.

After decoding, Chords-Web uses each sample as `[counter, CH1, CH2, ..., CHn]`. This is the same order as the columns in the recorded CSV file.

### 3.9.2 Checking for Dropped Samples

- **Counter**: In your recorded CSV file, the `Counter` value should go up by exactly 1 in each row (after 255 it goes back to 0). If it skips a number, samples were dropped.
- **Number of samples**: The total number of rows should be close to *sampling rate* × *recording time in seconds*. For example, a 10-minute recording at 500 Hz should have about 500 × 60 × 10 = 300,000 samples. A big difference means samples were dropped or the timing has drifted.

## 3.10 Troubleshooting

#### "Browser not supported" message

Use the latest version of Google Chrome, Microsoft Edge or Opera on a desktop/laptop. See *Browser Compatibility*.

#### My board doesn't appear in the port list

- Make sure you are using a USB **data** cable, not a charge-only cable.
- Close the Arduino IDE Serial Monitor/Plotter and any other app using the port.
- On Linux, add your user to the `uucp` (Arch/Manjaro) or `dialout` (Ubuntu/Debian) group (see *Connecting over Serial (USB)*).
- On Windows, some boards need a USB driver (see *Option 1: Update firmware from Chords-Web (Recommended)*).

#### "Your firmware is not updated" dialog appears

Your board doesn't have the latest Chords firmware. Click **Flash firmware** to update it (see *Option 1: Update firmware from Chords-Web (Recommended)*), or upload it with the Arduino IDE.

### My NPG Lite doesn't appear in the Bluetooth list

- Make sure the NPG Lite is switched on and has the **BLE firmware** (upload it with the NPG Lite Flasher Web or the NPG Lite Flasher).
- Turn on Bluetooth on your computer.
- Make sure the NPG Lite isn't already connected to another device or browser tab.

### My signal is noisy

- Apply the 50 Hz or 60 Hz notch filter, and the filter for your signal type (see *Filters*).
- Unplug your laptop charger while recording.
- Check that the electrodes are placed correctly and have good skin contact.
- For EEG, see Troubleshooting EEG Signal Quality.

Have a question or found a bug? Raise an issue on the Chords-Web GitHub repository.

## 3.11 Technologies Used

Chords-Web also uses TypeScript, Lucide icons, the Web Bluetooth API (for the NPG Lite) and the WebUSB API (for firmware updates).

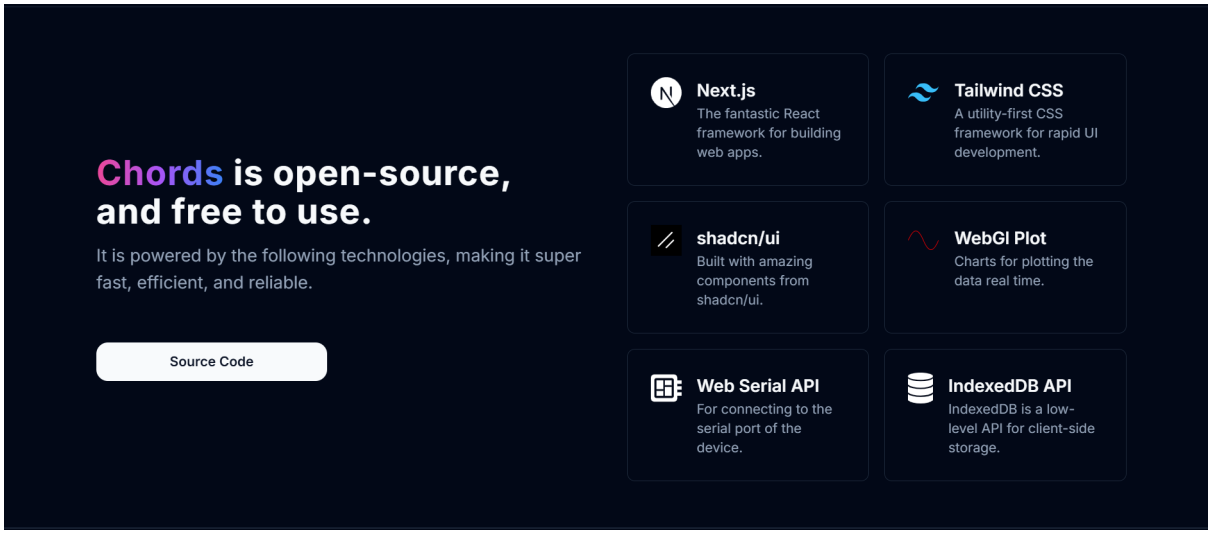



Fig. 17: Chords-Web Tech Stack

# 4 Chords-Python

https://youtu.be/bj7exKRsuZ8

## 4.1 Overview

Chords-Python is an open-source bag of tools for recording biopotential signals like **ECG**, **EMG**, **EEG**, or **EOG**, along with visualization using BioAmp hardware. It's ideal for educational purposes as it promotes DIY Neuroscience and makes biopotential signal exploration more accessible for students and researchers.

## 4.2 Features

| Feature | Description |
|---|---|
| Connection | Supports both wired and wireless connections via Wi-Fi, Bluetooth, or USB (Serial). |
| Data Reading | Reads data packets from development board in real-time, efficiently processing them to prevent data loss and ensure accurate signal representation for analysis and visualization. |
| CSV Logging | Optionally logs incoming data to a CSV file with columns for counter and channel data, enabling easy storage, analysis, and sharing. |
| LSL Streaming | Streams data via Lab Streaming Layer (LSL), a protocol for time-synchronized data sharing. Enables real-time analysis, visualization, or integration with tools like BrainVision. |
| Applications | Provides an interface to run multiple applications simultaneously via the LSL stream. |
| Create Custom Application | Allows users to develop and integrate their own applications with the system. |

## 4.3 Software Requirements

- Arduino IDE – Required to upload the Chords Arduino firmware to your development board.
- Python – Ensure you have the most recent version installed.
- VS Code or any other code editor (Alternatively, you can use the Command Prompt).

## 4.4 Hardware Requirements

To use Chords-Python, you need:

- A development board (Compatible Boards)
- A USB cable (type depends on the board)
- BioAmp Hardware and its accessories

## 4.5 Setting up the hardware

Make all the connections according to the hardware you are using , including sensor connections with the development board, body connections with the sensor, and connections from the development board to your laptop.

## 4.6 Uploading the code

Once you are all set, it is time to upload the code to your development board. Go to Chords Arduino Firmware GitHub repo, scroll down to the supported boards table, find your board name, and click on the Arduino sketch corresponding to that row. Copy the sketch and paste it into Arduino IDE. Go to tools, select your board, and the correct COM port. Now, hit the upload button.

## 4.7 Opening Chords-Python

There are two ways to use Chords-Python:

**A. Using the `chordspy` Python Package (Recommended)**

**B. Running Scripts Manually from the Repository**

### 4.7.1 A. Using the chordspy Package

This is the smoothest way to get started. Follow the steps below:

1. **Install Python** Make sure latest version of Python is installed.
2. **Create and Activate a Virtual Environment**:
   - **On Windows**:

     ```
     python -m venv .venv
     ```

     ```
     .venv\Scripts\activate
     ```

   - **On macOS/Linux**:

     ```
     python3 -m venv .venv
     ```

     ```
     source .venv/bin/activate
     ```

3. **Install the Package**:

   ```
   pip install chordspy
   ```

4. **Launch the Web Interface**:

   ```
   chordspy
   ```

A web interface will open where you can connect your device and access applications.

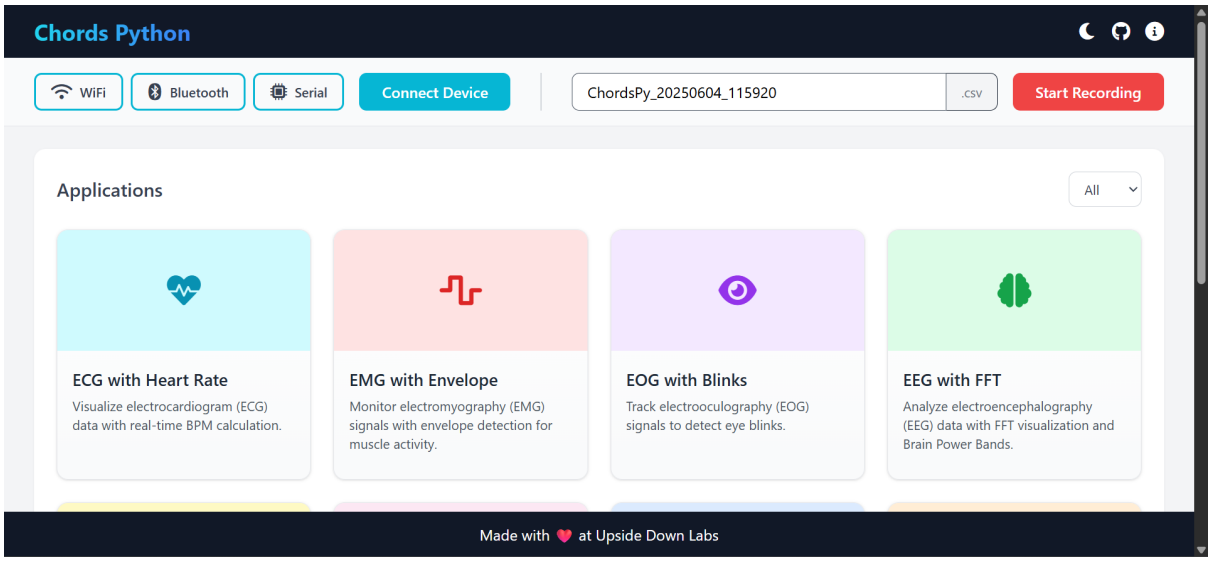



Fig. 1: Interface in Light Mode

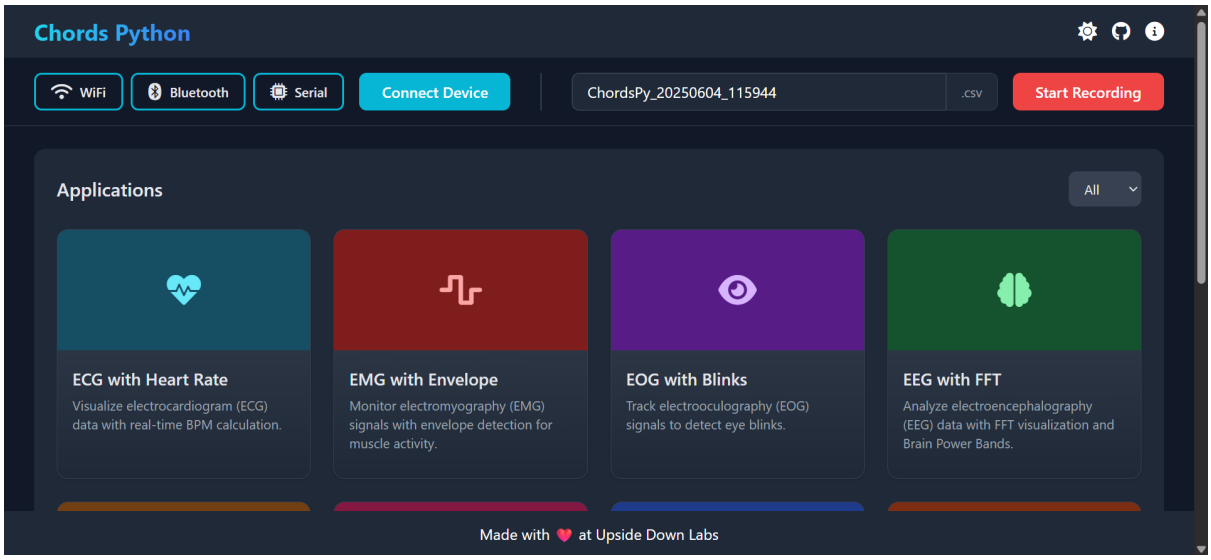



Fig. 2: Interface in Dark Mode

### 4.7.2 B. Running Scripts Manually (Alternative)

If you prefer running scripts directly (for development, debugging, or customization):

1. **Download the Repository**:
   - You can download the Chords-Python repository from GitHub by visiting the following link: Chords-Python.
   - Or, you can clone the repository using Git by running the following command:

```
git clone https://github.com/upsidedownlabs/Chords-Python.git
```

2. **Create and Activate a Virtual Environment** (if not already):
   - **On Windows**:

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

   - **On macOS/Linux**:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

3. **Install Requirements**:

```
pip install -r requirements.txt
```

4. **Run the Application**:

   Navigate to the *chordspy* folder and run:

```
python -m chordspy.app                          # To launch the web interface
python -m chordspy.connection --protocol usb    # To start LSL stream via USB
python -m chordspy.connection --protocol ble    # To start LSL stream via BLE
python -m chordspy.connection --protocol wifi   # To start LSL stream via WiFi
```

   To run any application, open a new terminal:

```
python chordspy.gui.py          # GUI Application
python chordspy.ffteeg.py       # EEG with FFT Analysis
```

## 4.8 Connection

The first step is to establish a connection with your device and start the stream.

There are three connection options available:

- Wi-Fi
- Bluetooth
- Serial (USB)

### 4.8.1 Wi-Fi

1. Upload the Wi-Fi firmware through the `Chords-Arduino-Firmware` repository
2. Turn on the device and connect to the access point created by the device (e.g., `npg-lite-2`)
3. In the web interface:
    - Click the **Wi-Fi** button
    - Click the **Connect** button

A pop-up notification will appear indicating a successful connection.

### 4.8.2 Bluetooth

1. Upload the Bluetooth firmware through the `Chords-Arduino-Firmware` repository
2. Turn on the device and enable Bluetooth on your computer
3. In the web interface:
    - Click the **Bluetooth** button
    - Select your device from the list of available devices
    - Hit the **Connect** button

A pop-up notification will appear indicating a successful connection.

### 4.8.3 Serial (USB)

1. Upload the Serial firmware through the `Chords-Arduino-Firmware` repository
2. Connect the device to your computer using a USB cable
3. In the web interface:
    - Click the **Serial** button
    - Click the **Connect** button

A pop-up notification will appear indicating a successful connection.

> **Note**
>
> The connection step is essential as it initiates the LSL Stream, which is required for running applications.

## 4.9 CSV Logging

The raw data received from the device can be logged to a CSV file for further analysis or record-keeping. This optional feature can be enabled or disabled in the web interface.

To use CSV logging:

1. Click the **Start recording** button to begin logging - A file with name is created `ChordsPy_{timestamp}.csv` in the same folder. - File includes columns for counter and channel data
2. Click the **Stop recording** button to end logging - File will be saved in the same folder

Fig. 3: CSV Logging

## 4.10 Applications

There are many applications available that stream the LSL and can be run for various purposes.

List of available applications:

### 4.10.1 1. *ECG with Heart Rate*

https://youtu.be/tZud2tc-TGI

#### Overview

The **ECG with Heart Rate** is a real-time application designed to visualize and analyze Electrocardiogram (ECG) data using the Lab Streaming Layer (LSL) protocol. Built with Python and PyQt5, this application provides a graphical interface for monitoring ECG signals, detecting R-peaks (heartbeats), and calculating the heart rate in real time. It applies signal processing techniques and utilizes the *neurokit2* library to estimate R-peak detection and heart rate.

### Features

| Features | Description |
|---|---|
| 1. Real-Time ECG Visualization | • Displays real-time ECG signals in a dynamic plot using *pyqtgraph*.<br>• Supports adjustable y-axis scaling based on the sampling rate (e.g. 250 Hz or 500 Hz). |
| 2. R-Peak Detection | • Utilizes the *neurokit2* library to detect R-peaks in the ECG signal.<br>• Highlights detected R-peaks as red dots on the plot for easy visualization. |
| 3. Heart Rate Calculation | • Computes heart rate (in BPM) using the time intervals between consecutive R-peaks.<br>• Implements a moving average filter to smooth heart rate values and reduce noise. |
| 4. Interactive GUI | • Built with PyQt5, providing a user-friendly interface with a real-time ECG plot and heart rate display.<br>• Allows double-click to reset the plot's zoom level to default settings. |
| 5. Signal Filtering | • Applies a low-pass Butterworth filter to remove high-frequency noise from the ECG signal.<br>• Helps in enhancing signal clarity, which can assist in identifying R-peaks. |

A GUI window will appear, displaying the real-time ECG signal along with the calculated heart rate.

## 4.10.2 2. *EMG with Envelope*

https://youtu.be/TiDwSQEY2eY

### Overview

The **EMG with Envelope** is a Python-based application designed to visualize and analyze Electromyography (EMG) signals in real-time. It connects to an EMG data stream using the Lab Streaming Layer (LSL) protocol, processes the signal to extract the EMG envelope, and displays both the filtered EMG signal and its envelope in a user-friendly graphical interface. Built with *PyQt5* and *pyqtgraph*, the application provides a responsive and interactive visualization tool for students, researchers, or developers working with EMG data.

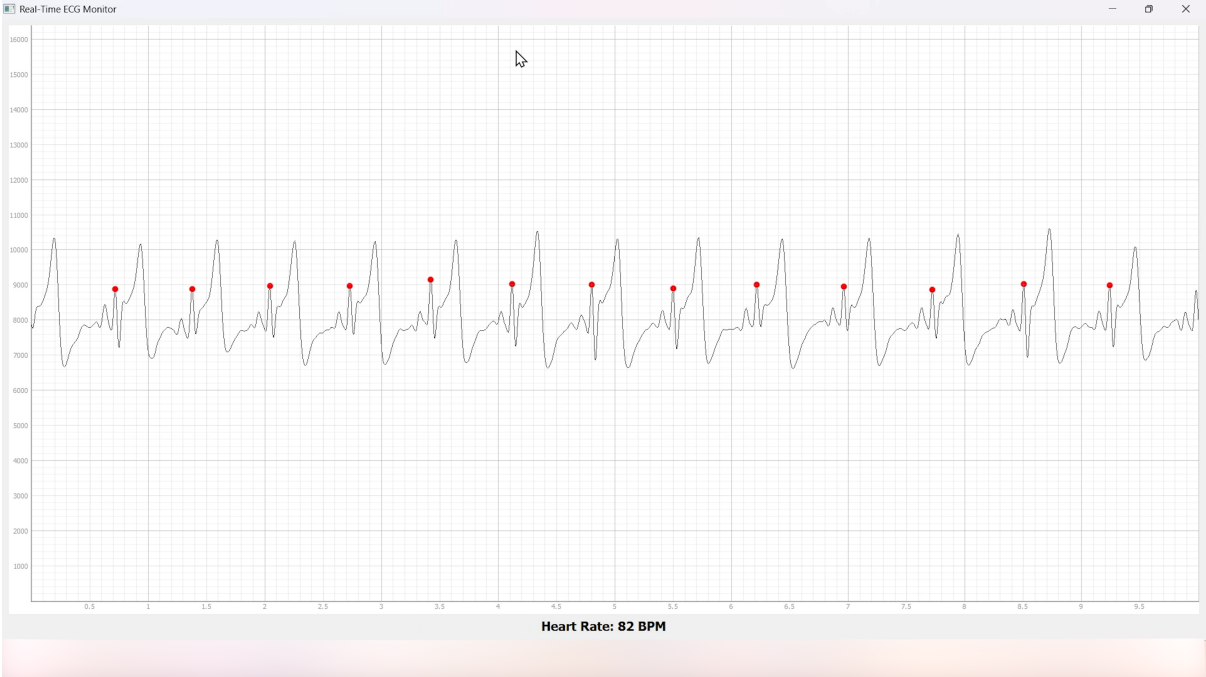



Fig. 4: Heart Rate with ECG

### Features

| Features | Description |
|---|---|
| 1. Real-Time EMG Signal Visualization | • Connects to an LSL stream to acquire real-time EMG data.<br>• Displays the EMG signal after applying a high-pass filter (70 Hz cutoff) to remove low-frequency noise. |
| 2. EMG Envelope Extraction | • Computes the Root Mean Square (RMS) envelope of the filtered EMG signal using a moving window.<br>• Applies convolution with a uniform window and pads the result to align with the original signal length. |
| 3. Interactive and Responsive GUI | • Built using *PyQt5* for a modern and intuitive user interface.<br>• Features two synchronized plots: one for the filtered EMG signal and one for the EMG envelope.<br>• Disables zoom and pan for a clean, fixed-axis visualization. |
| 4. Customizable Signal Processing | • Implements a high-pass Butterworth filter to remove baseline drift and noise.<br>• Adjusts the RMS window size dynamically based on the sampling rate (e.g., 25 samples for 250 Hz, 50 samples for 500 Hz). |
| 5. Dynamic Plot Updates | • Updates the plots in real-time using a fixed-size circular buffer for efficient data handling.<br>• Refreshes the display every 15 milliseconds for smooth and responsive visualization. |

A GUI window will appear, displaying the real-time EMG signal along with the calculated EMG Envelope.

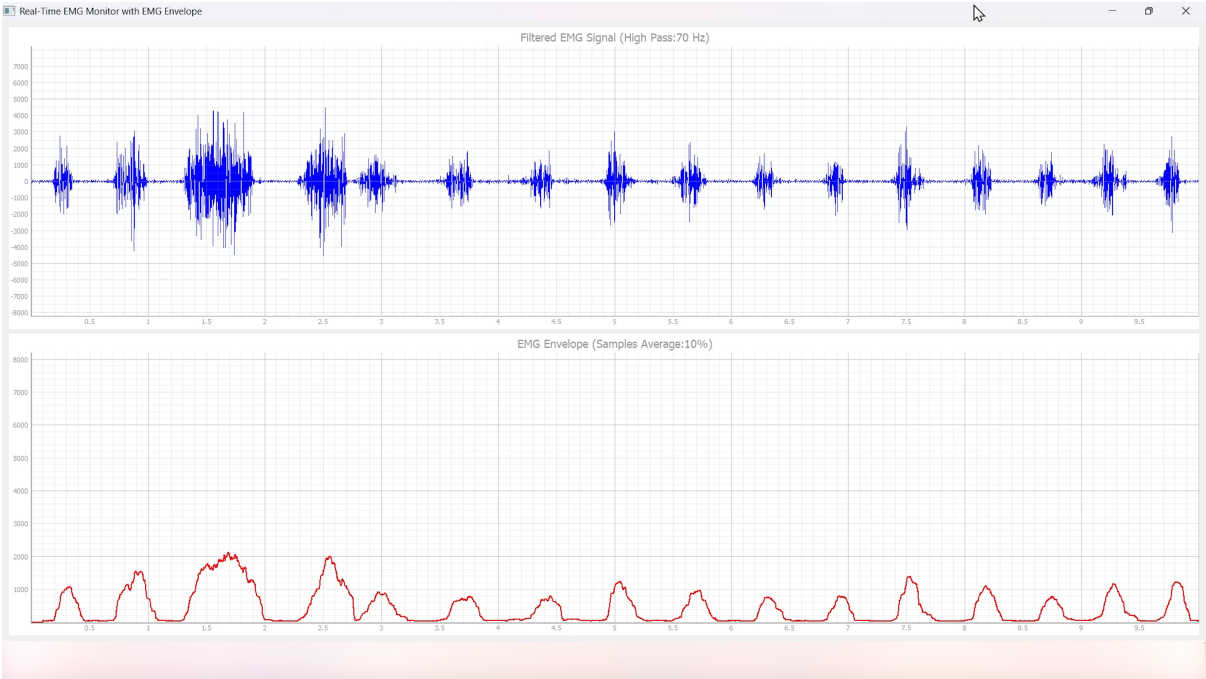



Fig. 5: EMG with Envelope

### 4.10.3 3. *EOG with Blinks*

The **EOG with Blinks** is a Python-based application designed to visualize and detect eye blinks in real-time using Electrooculography (EOG) signals. Built with the PyQt5 framework and PyQtGraph for plotting, the application connects to an LSL (Lab Streaming Layer) stream to acquire EOG data, processes the signal using a low-pass filter, and detects blinks based on dynamic thresholds. The application provides a dual-plot interface to display the filtered EOG signal and detected blinks, making it a useful tool for real-time monitoring and analysis of EOG data.

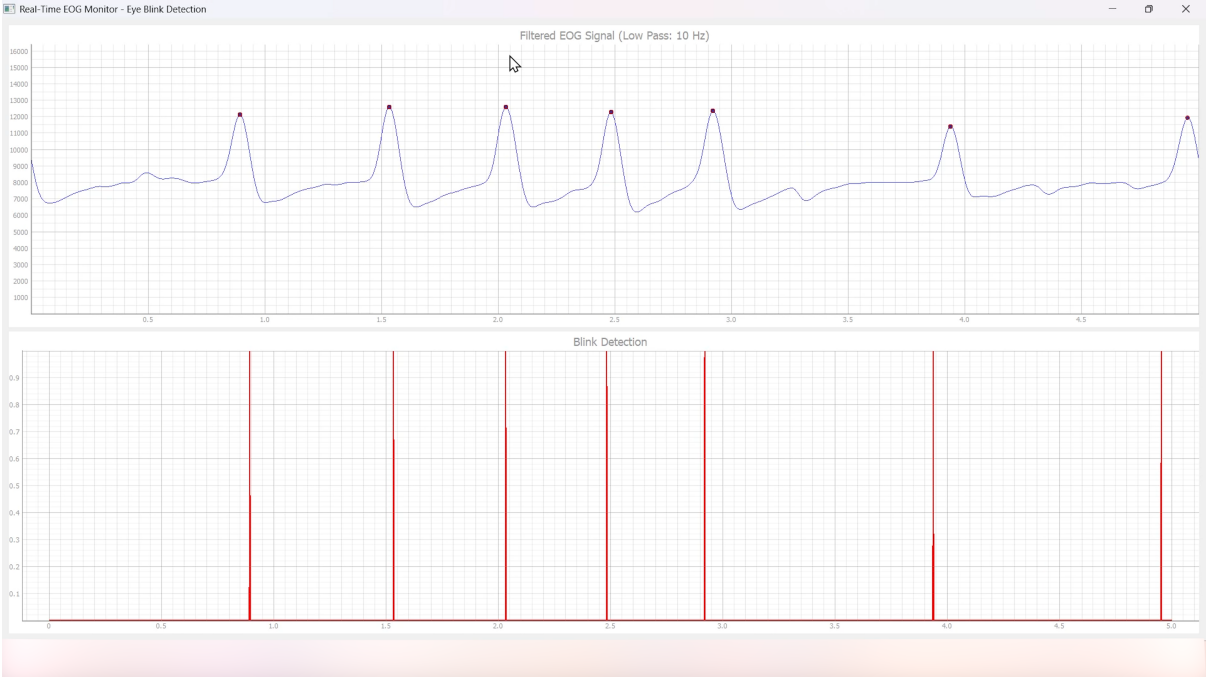



Fig. 6: EOG with Blinks

### 4.10.4 Features

| Features | Description |
|---|---|
| 1. Real-Time EOG Signal Visualization | • Displays the filtered EOG signal in real-time using a low-pass filter (10 Hz cutoff).<br>• Dynamically updates the plot with a 5-second rolling window for continuous monitoring. |
| 2. Dual-Plot Interface | • EOG Signal Plot: Displays the filtered EOG signal with detected peaks marked in red.<br>• Blink Detection Plot: Shows a binary representation of detected blinks (1 for blink, 0 for no blink). |
| 3. Blink Detection | • Detects blinks by identifying peaks in the filtered EOG signal.<br>• Uses a dynamic threshold based on the mean and standard deviation of the signal to distinguish blinks from noise.<br>• Implements a minimum time gap (0.1 seconds) between detected blinks to avoid false positives. |
| 4. User-Friendly GUI | • Built with PyQt5 for a responsive and intuitive interface.<br>• Includes features like grid lines, auto-scaling, and zoom disablement for better usability. |

A GUI window will appear, displaying the real-time EOG signal along with the Blinks marked as Red dot.

### 4.10.5 4. *EEG with FFT*

https://youtu.be/yVD9KmyZgxA

#### Overview

The **EEG with FFT and Brainwave Power** is a Python-based application designed to visualize and analyze Electroencephalography (EEG) signals in real-time. It connects to an EEG data stream using the Lab Streaming Layer (LSL) protocol, processes the signal to remove noise, and performs Fast Fourier Transform (FFT) to compute the power of different brainwave frequency bands (Delta, Theta, Alpha, Beta, and Gamma). The application provides a graphical user interface (GUI) built with *PyQt5* and *pyqtgraph* for real-time visualization of raw EEG signals, FFT results, and brainwave power distribution.

### Features

| Features | Description |
|---|---|
| 1. Multi-Channel EEG Visualization | • Displays raw EEG signals from all available channels in real-time.<br>• Each channel shown in a scrolling plot with 500-sample moving window. |
| 2. Multi-Channel FFT Analysis | • Computes and displays FFT for all EEG channels simultaneously.<br>• Visualizes the FFT results in a separate plot, focusing on the 0-50 Hz range. |
| 3. Signal Processing | • Applies a notch filter to remove 50 Hz powerline interference.<br>• Uses a bandpass filter (0.5-48 Hz) to isolate relevant EEG frequencies.<br>• Implements a Hanning window for FFT computation to reduce spectral leakage. |
| 4. Single-Channel Brainwave Power Analysis | • **Calculates the power of five brainwave frequency bands:**<br>– Delta (0.5-4 Hz)<br>– Theta (4-8 Hz)<br>– Alpha (8-13 Hz)<br>– Beta (13-30 Hz)<br>– Gamma (30-45 Hz)<br>• Displays the power of each band in a bar chart for easy comparison. |
| 5. User-Friendly GUI | • **Provides a clean and intuitive interface with Three-panels:**<br>– Top-left: Multi-channel EEG waveforms<br>– Top-right: Multi-channel FFT results<br>– Bottom-right: Single-channel brainwave power analysis<br>• Allows users to monitor multi-channel EEG data and its frequency components simultaneously. |

A GUI window will appear, displaying the real-time EEG signal along with the calculated FFT and Brainwave power distribution.

**Tip**

To ensure you're recording a high-quality signal, refer to the detailed guide here: Troubleshooting EEG Signal Quality.

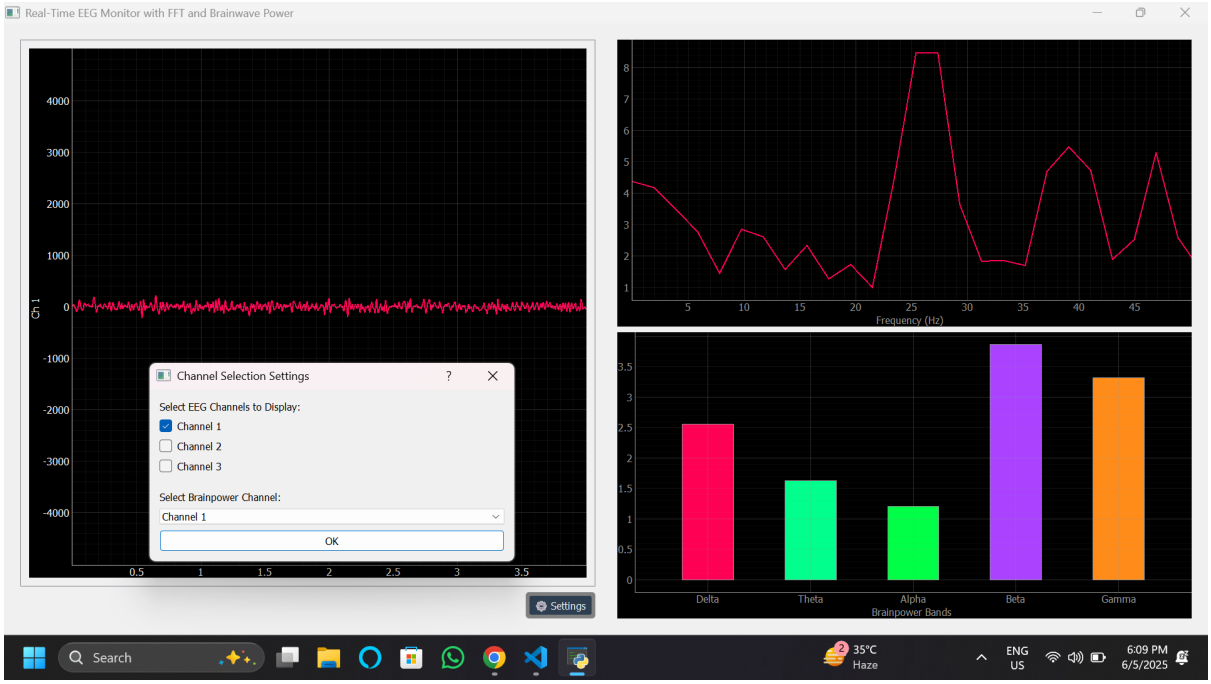



Fig. 7: EEG with FFT

### 4.10.6 5. *EEG Tug of War Game*

https://youtu.be/XAhcYg1J_7k

#### Overview

The **EEG Tug of War Game** is a Python-based application that leverages Electroencephalography (EEG) signals to create an interactive two-player game. Players control the movement of a ball on the screen by modulating their brain activity, specifically the Alpha and Beta frequency bands. The game uses the Lab Streaming Layer (LSL) protocol to acquire real-time EEG data, processes the signals to calculate relative power in the Alpha and Beta bands, and translates these into forces that move the ball. The first player aims to push the ball onto the opponent's side to score and win the game. The application is built using the *pygame* library for the graphical interface and integrates with *pylsl* for EEG data acquisition.

### Features

| Features | Description |
|---|---|
| 1. Real-Time EEG Signal Visualization | • Connects to an LSL stream to acquire real-time EEG data.<br>• Computes the power spectral density (PSD) of Alpha (8-13 Hz) and Beta (13-30 Hz) frequency bands using Welch's method.<br>• Calculates the relative power ratio (Beta/Alpha) to determine player force. |
| 2. Interactive Gameplay | • Two players compete to move a ball to the opponent's side using their brain activity.<br>• The ball's movement is determined by the net force derived from the players' EEG signals. |
| 3. Dynamic Thresholding | • Uses a moving average of the last 10 data points to smooth the force calculations.<br>• Applies a threshold to prevent small fluctuations from affecting the ball's movement. |
| 4. User-Friendly GUI | • Features a full-screen graphical interface with a central ball and two player paddles.<br>• Displays real-time updates of the ball's position and forces applied by each player.<br>• Includes buttons for starting, pausing, resuming, and exiting the game. |
| 5. Win Condition and Feedback | • Declares a winner when the ball reaches either side of the screen.<br>• Plays a sound effect to celebrate the winner.<br>• Automatically pauses the game upon a win and allows for a restart. |

The game window will open, featuring buttons for **START/RESTART**, **PLAY/PAUSE**, and **EXIT**. These buttons offer intuitive control, allowing players to easily start, pause, resume, or exit the game as needed.

For detailed instructions, check out the EEG Tug of War Game Instructable.

## 4.10.7 6. *EEG Beetle Game*

### Overview

The **EEG Beetle Game** is a Python-based application that uses Electroencephalography (EEG) signals to control a beetle's movement in a 2D game environment. The game leverages the Lab Streaming Layer (LSL) protocol to acquire real-time EEG data, processes the signal to detect the user's focus level, and translates it into upward or downward movement of the beetle. The application is built using the *pygame* library for the game interface and integrates signal processing techniques to analyze EEG data in real-time.

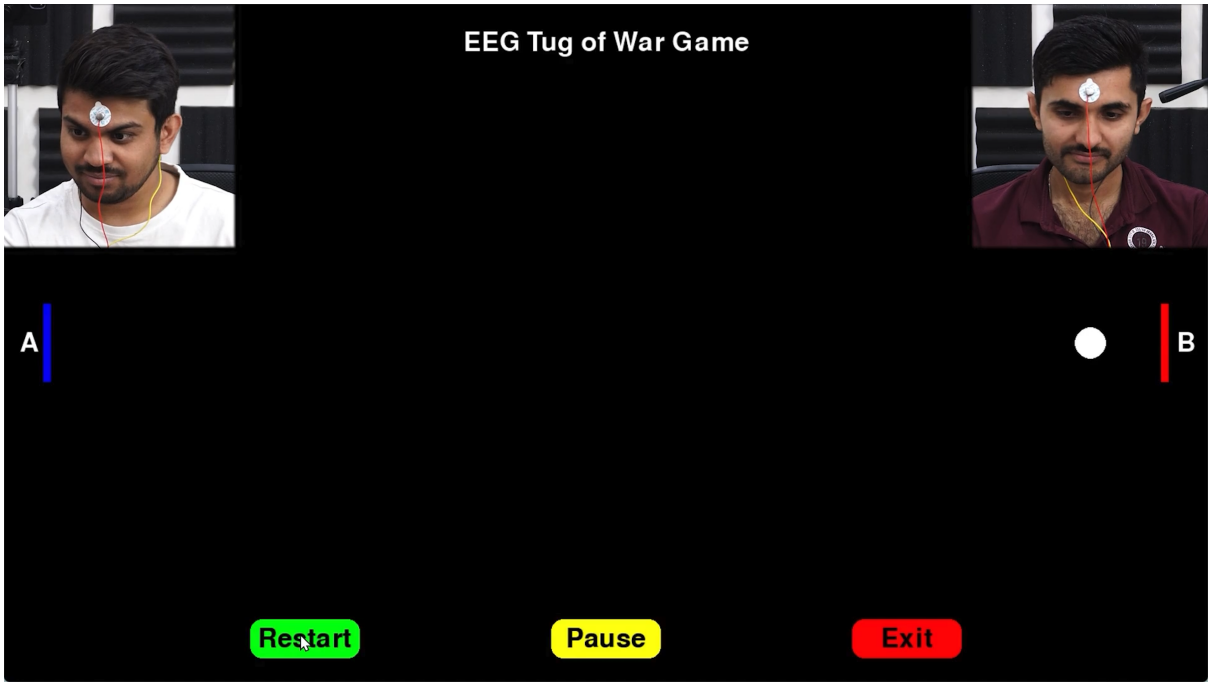



Fig. 8: EEG Tug of War

### Features

| Features | Description |
|---|---|
| 1. Real-Time EEG Signal Visualization | • Connects to an LSL stream to acquire real-time EEG data.<br>• Implements a notch filter to remove 50 Hz power line interference and a bandpass filter to isolate relevant EEG frequency bands (0.5–48 Hz). |
| 2. Focus Level Calculation | • Computes the user's focus level by analyzing the power spectral density of the EEG signal.<br>• Focus level is calculated using the ratio of high-frequency (beta and gamma) to low-frequency (delta, theta, and alpha) power bands. |
| 3. Calibration System | • Includes a calibration phase to establish a baseline focus level for the user.<br>• Dynamically sets a focus threshold based on the user's EEG data during calibration. |
| 4. Beetle Movement Control | • Moves the beetle upward when the user's focus level exceeds the threshold.<br>• Moves the beetle downward when the focus level is below the threshold.<br>• Implements smooth animation and boundary constraints to ensure the beetle stays within the game window. |
| 5. Interactive Game Interface | • Features a 2D game environment with a beetle sprite that responds to the user's focus level.<br>• Displays real-time feedback on the beetle's position and focus level. |
| 6. Dynamic Animation | • Uses a sequence of beetle sprites to create smooth animations.<br>• Adjusts animation speed based on the game's frame rate. |

A GUI window will appear, showing all calibration messages, followed by the game starting, and finally displaying the game with the beetle.

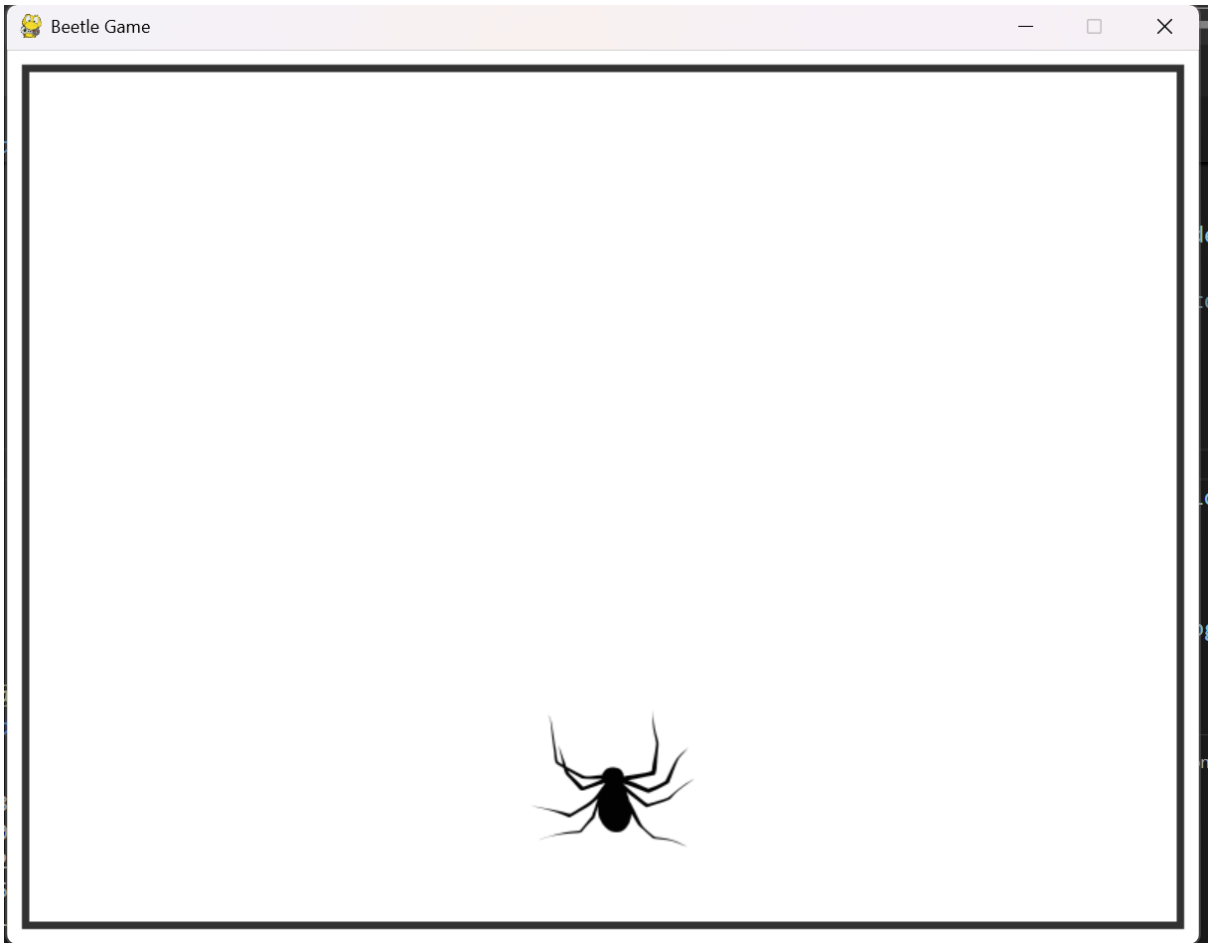



Fig. 9: EEG Beetle Game

### 4.10.8 7. *GUI*

https://youtu.be/BseTIdoimws

#### Overview

The **GUI** application is a Python-based tool designed to visualize real-time data streams from an Arduino device using the Lab Streaming Layer (LSL) protocol. The application connects to an LSL stream, retrieves multi-channel data, and plots it in real-time using the *pyqtgraph* library.

#### Features

| Features | Description |
|---|---|
| 1. LSL Stream Integration | • Automatically searches for and connects to available LSL streams.<br>• Supports dynamic detection of the number of channels in the stream.<br>• Displays connection status and channel count in the GUI. |
| 2. Real-Time Data Visualization | • Plots real-time data for each channel in separate graphs.<br>• Updates plots at a high frequency for smooth visualization. |
| 3. Customizable GUI | • Built using *PyQt* and *pyqtgraph* for a responsive and interactive interface.<br>• Features a clean layout with individual plots for each channel.<br>• Includes a status bar to display LSL connection details. |

A GUI window will appear that shows the data in real-time.

### 4.10.9 8. *EOG Keystroke Emulator*

https://youtu.be/ZJmUUtHJj08

#### Overview

The **EOG Keystroke Emulator** is a Python-based application designed to detect eye blinks using Electrooculography (EOG) signals and translate them into keystrokes. The application leverages the Lab Streaming Layer (LSL) protocol to acquire real-time EOG data, processes the signal to detect blinks, and simulates a spacebar press whenever a blink is detected. The application is built using the *tkinter* library for the graphical user interface (GUI) and integrates with *pyautogui* for keystroke emulation.

Real-Time Arduino Data
Channel 1
Channel 2
Channel 3
Channel 4
Channel 5
Channel 6
LSL Status: Connected

Fig. 10: GUI

### Features

| Features | Description |
|---|---|
| 1. Real-Time EOG Signal Processing | • Connects to an LSL stream to acquire real-time EOG data.<br>• Implements a low-pass filter to smooth the EOG signal for accurate blink detection. |
| 2. Blink Detection | • Detects blinks by identifying peaks in the filtered EOG signal.<br>• Uses a dynamic threshold based on the mean and standard deviation of the signal to distinguish blinks from noise.<br>• Incorporates a refractory period to prevent multiple detections from a single blink. |
| 3. Keystroke Emulation | • Simulates a spacebar press (*pyautogui.press('space')*) whenever a blink is detected.<br>• Provides visual feedback by updating the GUI button color upon blink detection. |
| 4. User-Friendly GUI | • Features a compact, movable popup window with a clean and intuitive interface.<br>• Includes buttons for connecting to the LSL stream, starting/stopping blink detection, and quitting the application.<br>• Displays an eye icon to represent the blink detection status. |

A small window appears in the corner, displaying a *Connect* button. Once connected, a *Start* button becomes visible. Pressing the *Start* button initiates blink detection, and each detected blink triggers a spacebar key press.

Fig. 11: Keystroke Emulator

### 4.10.10 9. *CSV Plotter*

https://youtu.be/wMnCOprRpZo

#### Overview

The **CSV Plotter** is a Python-based application designed to visualize data from CSV files. Built using the *tkinter* library for the graphical user interface (GUI) and *plotly* for data visualization, this tool allows users to load CSV files, select specific data channels, and generate interactive line plots.

#### Features

| Features | Description |
|---|---|
| 1. Load CSV Files | • Users can load CSV files containing data with a *Counter* column and multiple channels (e.g., *Channel1*, *Channel2*, etc.).<br>• The application automatically detects the header row and skips any metadata above it. |
| 2. Channel Selection | • A dropdown menu dynamically populates with available channels (e.g., *Channel1*, *Channel2*, etc.) from the loaded CSV file.<br>• Users can select a specific channel to plot. |
| 3. Interactive Data Visualization | • Utilizes *plotly* to generate interactive line plots for the selected channel.<br>• **Plots include advanced features such as:**<br>– *Zoom*: Zoom in to inspect specific data ranges.<br>– *Pan*: Move across the plot to explore different sections.<br>– *Autoscale*: Automatically adjust the plot scale to fit the data.<br>– *Download Plot as PNG*: Save the generated plot as a high-quality PNG image.<br>– *Hover-to-View Data Points*: Hover over the plot to view precise data values. |
| 4. User-Friendly Interface | • Simple and intuitive GUI with buttons for loading files and plotting data.<br>• Displays the name of the loaded CSV file for easy reference. |

A small pop-up will appear, providing options to load the file, select the channel, and plot the data.

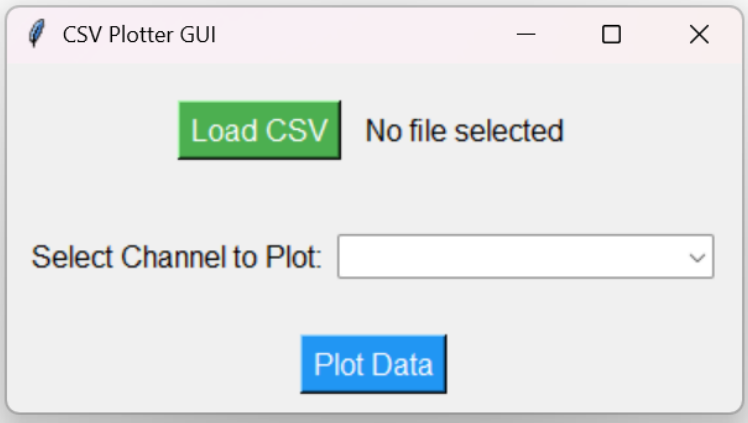



Fig. 12: CSV Plotter

### 4.10.11 10. *EOG Morse Decoder*

#### Overview

The **EOG Morse Decoder** is a Python-based application that enables users to input Morse code using eye movements detected via Electrooculography (EOG) signals. By moving your eyes left or right, you can generate dots and dashes, and by performing double or triple blinks, you can send characters or backspace. This application is ideal for hands-free communication and accessibility research.

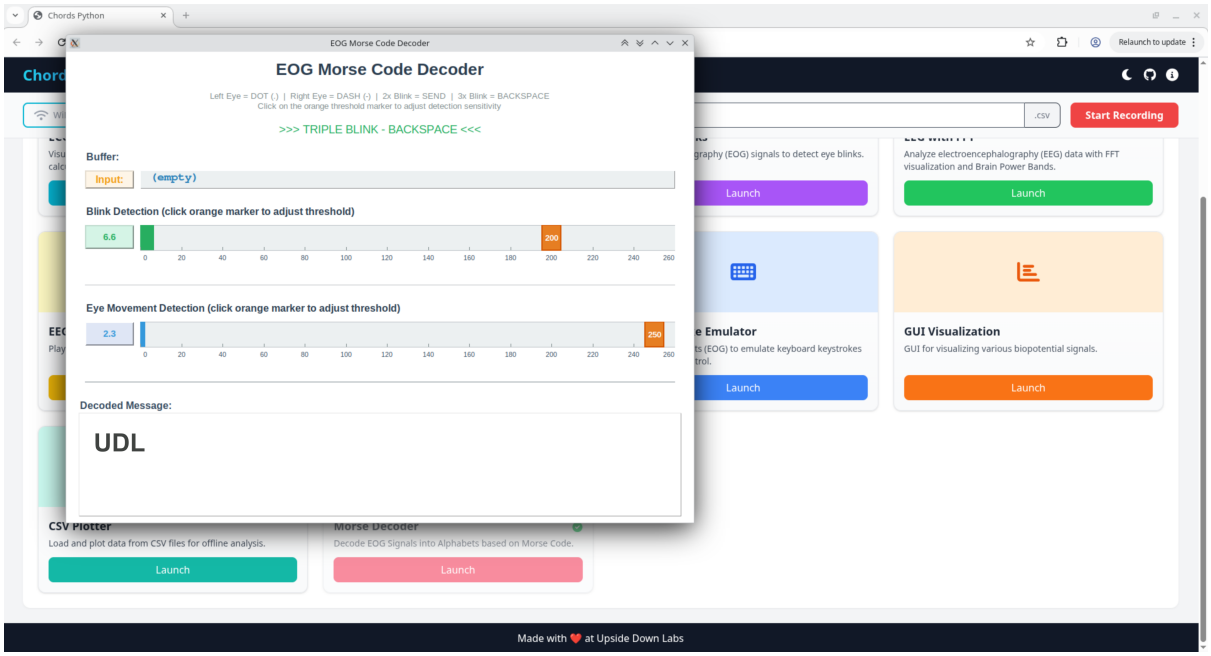



Fig. 13: Morse Decoder

### 4.10.12 Features

| Features | Description |
|---|---|
| 1. Real-Time EOG Signal Processing | • Connects to an LSL stream to acquire real-time EOG data.<br>• Applies notch and bandpass filters to remove noise and isolate eye movement signals. |
| 2. Eye Movement Detection | • Detects left and right eye movements by analyzing deviations from a dynamically calculated baseline.<br>• Uses adjustable thresholds with interactive GUI sliders for real-time sensitivity adjustment. |
| 3. Morse Code Input | • Left eye movement generates a dot (.)<br>• Right eye movement generates a dash (-)<br>• Double blink sends the current Morse sequence as a letter.<br>• Triple blink acts as backspace to delete the last character. |
| 4. Interactive GUI with Real-Time Visualization | • Displays the decoded message in a large, clear font using a Tkinter GUI.<br>• Real-time bar graphs showing blink envelope and eye movement deviation.<br>• Click-and-drag threshold adjustment on visual indicators.<br>• Shows current buffer contents and detection status. |
| 5. Adjustable Parameters | • Interactive threshold adjustment via clickable/draggable markers on visualization bars.<br>• Separate thresholds for blink detection and eye movement detection.<br>• Configurable debounce times for preventing false detections. |

A GUI window will appear that allows you to input Morse code using eye movements and displays the decoded text in real-time.

#### Electrode Placement

Proper electrode placement is crucial for accurate EOG signal detection. The table below shows the recommended electrode configuration:

| Electrode Pin | Description | Placement |
|---|---|---|
| A0P | Channel 1 Positive | Below eye |
| A0N | Channel 1 Negative | Above eye |
| A1P | Channel 2 Positive | On the left of left eye |
| A1N | Channel 2 Negative | On the right of right eye |
| REF | Reference | Bony part behind the ear |

**Note**

- Channel 1 (A0P/A0N) detects vertical eye movements for blink detection
- Channel 2 (A1P/A1N) detects horizontal eye movements for left/right detection
- Proper skin preparation and electrode contact quality are essential for optimal performance

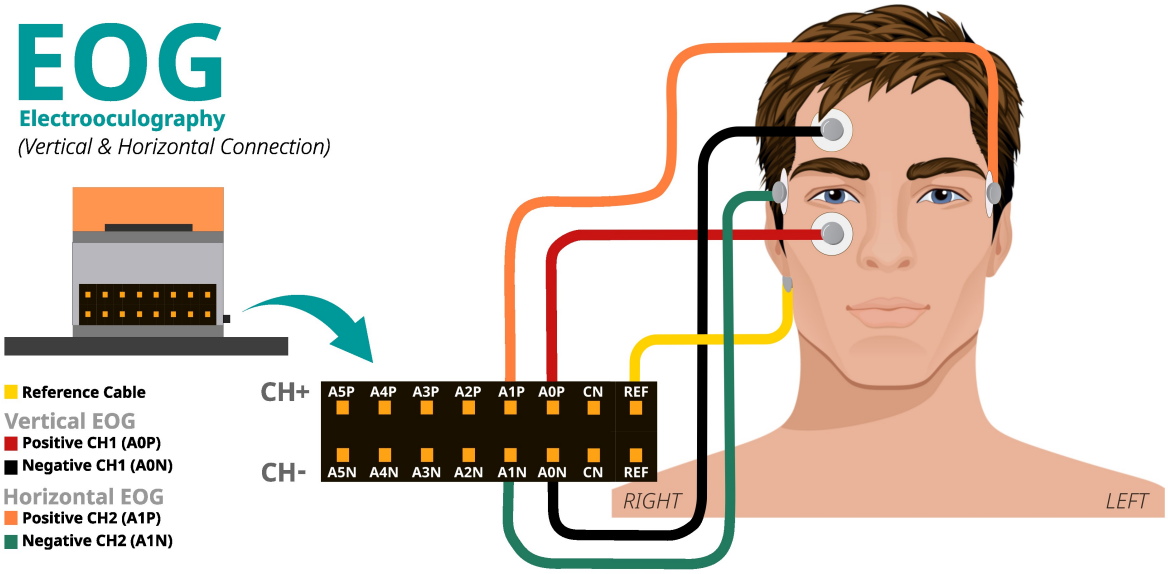



Fig. 14: Electrode Placement Diagram

## How It Works

- **Signal Acquisition**: The application connects to an LSL stream and continuously reads vertical and horizontal EOG data from two channels.
- **Filtering**: Notch filters (50Hz), high-pass filters (1Hz), and low-pass filters (10Hz) are applied to remove powerline interference and isolate relevant frequencies.
- **Baseline Tracking**: A rolling baseline is maintained for horizontal EOG to adapt to individual signal characteristics.
- **Envelope Detection**: Vertical EOG signal envelope is computed for reliable blink detection.
- **Movement Detection**: Horizontal EOG deviations from baseline are analyzed to classify left and right eye movements.
- **Morse Code Input**:
  - Left eye movement → Dot (.)
  - Right eye movement → Dash (-)
  - Double blink → Sends the current Morse sequence as a letter
  - Triple blink → Backspace (deletes last character)
- **GUI Display**: Real-time visualization shows detection thresholds, signal levels, buffer contents, and decoded message.

### Adjusting Detection for Your Neural Signals

The application provides an intuitive GUI for adjusting detection sensitivity in real-time:

**Interactive Threshold Adjustment:**

- **Blink Detection Threshold**: Click or drag the orange marker on the blink detection bar to adjust the threshold (range: 40-260). Higher values require stronger blinks.
- **Eye Movement Threshold**: Click or drag the orange marker on the eye movement bar to adjust the threshold (range: 40-260). Higher values require larger eye movements.

**Visual Feedback:**

- The blink detection bar shows the current envelope value (green when below threshold, red when above)
- The eye movement bar shows deviation from baseline (blue when below threshold, purple for left movement, red for right movement)
- Threshold markers display the current threshold value and can be dragged for adjustment

**Timing Parameters** (adjustable in code if needed):

- `BLINK_DEBOUNCE_MS` (default: 200ms): Minimum time between detecting separate blinks
- `DOUBLE_BLINK_MS` (default: 1000ms): Time window for double blink detection
- `TRIPLE_BLINK_MS` (default: 1500ms): Time window for triple blink detection
- `EYE_MOVEMENT_DEBOUNCE_MS` (default: 750ms): Minimum time between detecting separate eye movements

**Tips for Optimal Performance:**

- Make sure NPG Lite is charged and not connected to a charging laptop or an AC powered device
- Sit away from AC powered appliances
- Ensure proper skin preparation and electrode contact quality. Read the Skin Preparation Guide for detailed instructions.
- Start with default threshold values and adjust using the GUI sliders based on your signal strength
- Ensure proper electrode placement for clean vertical and horizontal EOG signals
- Maintain a neutral gaze position during baseline calculation at startup
- Use the real-time visualization bars to monitor signal quality and adjust thresholds accordingly

### Troubleshooting

- If movements are not detected reliably, use the GUI to adjust thresholds in real-time by clicking/dragging the orange markers
- Ensure electrodes are placed correctly: vertical EOG above/below one eye, horizontal EOG at outer corners of both eyes
- Remain relaxed and maintain a neutral gaze position during the initial baseline calculation
- Check the real-time visualization bars to ensure signals are being detected properly
- If blinks or movements are too sensitive, increase the respective threshold using the GUI sliders
- If blinks or movements are not sensitive enough, decrease the threshold using the GUI sliders

## 4.11 Create Custom application

You can create custom applications using the provided framework by following these steps:

1. Configure Application Metadata:

Edit the `apps.yaml` file in the `config` folder with your application details:

```
- title: "Your Application Title"
  icon: "path/to/your/icon.png"
  color: "your_hex_color"
  script: "path/to/{app_name}.py"
  description: "Brief description of your application"
  category: "Your Category"
```

Add this as a new entry in the YAML list. Replace all placeholders with your actual application details.

**Note**

- The `icon` path should be relative to the application root directory
- `color` should be in HEX format (e.g., "#FF5733")
- The `script` path should point to your Python file

2. Create application script:

Create a new Python script in the main directory with your application name. The script should contain:

- LSL stream connection handling to receive device data
- User interface components using PyQt5/PyQtGraph
- Data processing logic for incoming signals

**Tip**

Use the existing applications in the repository as reference implementations for: - lsl setup and data acquisition - Advanced UI layouts - Signal processing examples - Performance optimization